

JS代码安全防护原理

本章将从原理的角度讲解 JS 是如何进行混淆的。一般情况下,JS 混淆是把原本可读性比较高的代码,用另外一种或者几种代码进行替换,降低代码的可读性,但是执行效果又是等同的。本章的内容是学好 AST 混淆和还原 JS 代码的基础,后续在第 9 章介绍如何使用 AST 来自动化混淆 JS 代码,在第 10 和 11 章介绍如何使用 AST 来自动化还原 JS 代码。

6.1 常量的混淆原理



视频讲解

以下代码用于格式化时间。这段代码逻辑简单清晰,它是在 Date 的原型对象上,增加了一个 format 方法。当实例化一个 Date 对象后,就可以直接调用从 Date 原型对象上继承过来的 format 方法。

```
Date.prototype.format = function (formatStr) {  
    var str = formatStr;  
    var Week = ['日', '一', '二', '三', '四', '五', '六'];  
    str = str.replace(/yyyy|YYYY/, this.getFullYear());  
    str = str.replace(/MM/, (this.getMonth() + 1) > 9 ? (this.getMonth() + 1).toString() :  
    '0' + (this.getMonth() + 1));  
    str = str.replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() : '0' + this.  
    getDate());  
    return str;  
}  
console.log(new Date().format('yyyy-MM-dd'));  
//输出结果 2020-07-04
```

上述代码没有经过任何处理,任何查看脚本的开发者都可以清楚地理解本段代码。假如这是某网站开发人员编写的一段关键代码,那么在代码发布后,很容易被第三方破解利用,从而引发安全问题。因此学习 JS 代码的防护技术就显得格外重要了。

6.1.1 对象属性的两种访问方式

示例代码如下：

```
function People(name){
    this.name = name;
}
People.prototype.sayHello = function(){
    console.log('Hello');
}
var p = new People('xiaojianbang');

console.log( p.name );      //xiaojianbang
p.sayHello();              //Hello
console.log( p['name'] );   //xiaojianbang
p['sayHello']();            //Hello
```

这段代码较为简单,有一个名为 People 的构造函数,并在其原型对象上定义了一个名为 sayHello 的方法。当 var p = new People('xiaojianbang') 这行代码执行后,会产生一个对象赋值给 p。此时,想要访问 p 对象下的 name 属性,有以下两种方式:

(1) p.name: 这种方式中,name 是一个标识符,必须明确出现在代码中,不能加密和拼接。

(2) p['name']: 这种方式中,name 是一个字符串。既然是字符串,访问的时候就可以进行拼接和加密。在 JS 混淆中,一般会选择用这种方式来访问属性。

访问对象的方法也一样。因为对象的方法可以看作特殊的属性,它是一种值为函数的属性。6.1 节的代码可以转换为如下形式:

```
Date.prototype.format = function (formatStr) {
    var str = formatStr;
    var Week = ['日', '一', '二', '三', '四', '五', '六'];
    str = str['replace'](/yyyy|YYYY/, this['getFullYear']());
    str = str['replace'](/MM/, (this['getMonth']() + 1) > 9 ? (this['getMonth']() + 1)
    ['toString']() : '0' + (this['getMonth']() + 1));
    str = str['replace'](/dd|DD/, this['getDate']() > 9 ? this['getDate']() ['toString']() :
    '0' + this['getDate']());
    return str
}
console.log(new window['Date']() ['format']('yyyy-MM-dd'));
//输出结果 2020-07-04
```

Date 是 JS 中的内置对象。在 JS 中,很多内置对象都是 window 的属性,所以 JS 中的内置对象和客户端 JS 中的 DOM 对于 JS 的防护与逆向极为重要。代码中定义的全局变量都是全局对象 window 的属性,定义的全局函数都是全局对象 window 的方法。全局对象的属性或方法在调用时,可以省略全局对象名。例如,new window.Date() 等同于 new Date()。

由于这里要把 Date 变为字符串,因此前面就必须加 window。

6.1.2 十六进制字符串

改变对象属性的访问方式后,代码的阅读性仍然较高,要继续进行复杂化处理。因为 JS 中的字符串支持以十六进制形式表示,所以可以用十六进制形式代替原有的字符串。如 'yyyy-MM-dd', 可以表示成 '\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64'。其中字符 y 转换为字节,再用十六进制表示就是 0x79(0x79 就是字符 y 的 Hex 形式的 ASCII 码)。可以使用以下代码,完成十六进制字符串的转换。

```
function hexEnc(code) {
    for(var hexStr = [], i = 0, s; i < code.length; i++) {
        s = code.charCodeAt(i).toString(16);
        hexStr += "\\x" + s;
    }
    return hexStr
}
```

在 JS 中, charAt 方法用来取出字符串中对应索引的字符。而 charCodeAt 方法用来取出字符串中对应索引的字符的 ASCII 码;然后用 toString(16) 转换为十六进制,再与 \x 拼接。为了方便理解,这里只处理一个字符串,代码转换为:

```
Date.prototype.format = function (formatStr) {
    var str = formatStr;
    var Week = ['日', '一', '二', '三', '四', '五', '六'];
    str = str['replace'](/yyyy|YYYY/, this['getFullYear']());
    str = str['replace'](/MM/, (this['getMonth']() + 1) > 9 ? (this['getMonth']() + 1)
    ['toString']() : '0' + (this['getMonth']() + 1));
    str = str['replace'](/dd|DD/, this['getDate']() > 9 ? this['getDate']()['toString']() :
    '0' + this['getDate']());
    return str
}
console.log(new window['Date']()['format']('\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64'));
//输出结果 2020-07-04
```

这种混淆方式很容易被还原,不会大量应用,只用在无法加密的字符串上。至于哪些字符串无法加密,在后续内容中会介绍。

十六进制字符串的还原方法很简单,把字符串放到控制台中输出即可。

6.1.3 unicode 字符串

在 JS 中,字符串除了可以表示成十六进制的形式以外,还支持用 unicode 形式表示。下面介绍两个转换的例子。

(1) 以 var Week = ['日', '一', '二', '三', '四', '五', '六'] 为例,可以表示成 var

```
Week = ['\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94', '\u516d']。
```

(2) 如果为非中文的情况,以'Date'为例,可以表示成 '\u0044\u0061\u0074\u0065'。

可以看出,unicode 的形式是:\u 开头,后跟四位数的十六进制数,不足四位的补 0。可以使用以下代码完成 unicode 转换。

```
function unicodeEnc(str) {
    var value = '';
    for (var i = 0; i < str.length; i++)
        value += "\\u" + ("0000" + parseInt(str.charCodeAt(i)).toString(16)).substr(-4);
    return value;
}
```

先取出字符串中对应索引字符的 ASCII 码,与'0000'拼接,取后四位,然后在前面补上\u。需要注意的是,JS 中的字母和中文都是一个字符。

```
console.log( 'x'.length );    // 1
console.log( '小'.length );   // 1
```

JS 中的标识符也支持以 unicode 形式表示。因此,代码中的 format、formatStr、str、

Week、window 等都支持以 unicode 形式表示。为了方便观察,这里只处理部分代码:

```
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {
    var \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
    var Week = ['\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94', '\u516d'];
    str = str['replace'](/yyyy|YYYY/, this['getFullYear']());
    str = str['replace'](/MM/, (this['getMonth']() + 1) > 9 ? (this['getMonth']() + 1)
    ['toString']() : '0' + (this['getMonth']() + 1));
    str = str['replace'](/dd|DD/, this['getDate']() > 9 ? this['getDate']() ['toString']() :
    '0' + this['getDate']());
    return str;
}
console.log( new \u0077\u0069\u006e\u0064\u006f\u0077['\u0044\u0061\u0074\u0065']()
    ['format']('\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64') );
//输出结果 2020 - 07 - 04
```

在使用\u0073\u0074\u0072定义变量后,依然能够使用对应的 str 来引用变量。在实际 JS 混淆的应用中,标识符一般不会替换成 unicode 形式,因为要还原它十分容易。通常的混淆方式是替换成没有语义,但看上去又很相似的名字,如_0x21dd83、_0x21dd84 和 _0x21dd85,或是由大写字母 O、小写字母 o 以及数字 0 组成的名字,Oo00Oo0、Oo00Oo0 和 oO000Oo,注意标识符不允许以数字开头。9.2.6 节中将介绍如何使用 AST 实现标识符混淆。

最后,介绍 unicode 字符串的还原方法。与十六进制字符串一样,把字符串放到控制台输出即可。

6.1.4 字符串的 ASCII 码混淆

为了完成字符串的 ASCII 码混淆,这里需要使用两个函数,一个是 String 对象下的 charCodeAt 方法,另一个是 String 类下的 fromCharCode 方法。先介绍这两个方法的使用。在控制台执行以下代码:

```
console.log( 'x'.charCodeAt(0) );           //120
console.log( 'b'.charCodeAt(0) );           //98
console.log( String.fromCharCode(120, 98) ); // "xb"
```

可以看出,这两个方法是相反的两个过程。String.fromCharCode 接收的是可变长度的数值类型的参数。使用以下代码,把一个字符串转换为字节数组:

```
function stringToByte(str){
    var byteArr = [];
    for(var i = 0; i < str.length; i++){
        byteArr.push(str.charCodeAt(i));
    }
    return byteArr;
}
console.log( stringToByte('xiaojianbang') );
// [120, 105, 97, 111, 106, 105, 97, 110, 98, 97, 110, 103]
```

接下来介绍字符串的 ASCII 码混淆方法,在 6.1.3 节转换后的代码中任意挑选一个字符串进行尝试。'format'转换为字节数组是[102, 111, 114, 109, 97, 116]。因此代码中的'format'字符串,可以表示为 String.fromCharCode(102, 111, 114, 109, 97, 116)。

注意,fromCharCode 接收的参数类型并非数组。如果想要传递数组,可以使用 String.fromCharCode.apply(null, [102, 111, 114, 109, 97, 116])。在 JS 中,函数也是对象,可以给函数定义属性和方法,而且函数本身就自带一些属性和方法。apply 就是从函数的原型对象 Function.prototype 继承过来的方法。

ASCII 码混淆不仅用来做字符串混淆,还可以用来做代码混淆。以下面这段代码为例,将其变为字符串,再转换成字节数组:

```
//str = str['replace'](/yyyy|YYYY/, this['getFullYear']());
stringToByte("str = str['replace'](/yyyy|YYYY/, this['getFullYear']());")
//[115, 116, 114, 32, 61, 32, 115, 116, 114, 91, 39, 114, 101, 112, 108, 97, 99, 101, 39, 93,
40, 47, 121, 121, 121, 121, 124, 89, 89, 89, 89, 47, 44, 32, 116, 104, 105, 115, 91, 39, 103,
101, 116, 70, 117, 108, 108, 89, 101, 97, 114, 39, 93, 40, 41, 41, 59]
```

这段字节数组可以通过 String.fromCharCode 转换成字符串。但转换以后只是一个字符串,不能当作代码来执行。于是,这里要引出另外两个函数 eval 与 Function。在 JS 中,要把字符串当作代码来执行,就需要用到这两个函数。其中 eval 用来执行一段代码,Function 用来生成一个函数。例如,在控制台执行 eval('var a = 1000; console.log(a)')会

输出 1000。处理后的代码为：

```
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {
    var \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
    var Week = ['\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94', '\u516d'];
    eval(String.fromCharCode(115, 116, 114, 32, 61, 32, 115, 116, 114, 91, 39, 114, 101, 112,
108, 97, 99, 101, 39, 93, 40, 47, 121, 121, 121, 121, 124, 89, 89, 89, 89, 47, 44, 32, 116, 104,
105, 115, 91, 39, 103, 101, 116, 70, 117, 108, 108, 89, 101, 97, 114, 39, 93, 40, 41, 41, 59));
    str = str['replace'](/MM/, (this['getMonth']() + 1) > 9 ? (this['getMonth']() + 1)
['toString']() : '0' + (this['getMonth']() + 1));
    str = str['replace'](/dd|DD/, this['getDate']() > 9 ? this['getDate']()['toString']() :
'0' + this['getDate']());
    return str;
}
console.log( new \u0077\u0069\u006e\u0064\u006f\u0077[ '\u0044\u0061\u0074\u0065' ] )
[String.fromCharCode(102, 111, 114, 109, 97, 116)]('\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64') );
//输出结果 2020 - 07 - 04
```

6.1.5 字符串常量加密

字符串常量加密的核心思想是，先把字符串加密得到密文，然后在使用前，调用对应的解密函数去解密，得到明文。代码中仅出现解密函数和密文。当然，也可以使用不同的加密方法去加密字符串，再调用不同的解密函数去解密。本节将把代码中剩下的字符串都处理完，字符串加密方式采用最简单的 Base64 编码。

replace	Base64 编码后为	cmVwbGFjZQ==
getMonth	Base64 编码后为	Z2V0TW9udGg=
getDate	Base64 编码后为	Z2V0RGF0ZQ==
0	Base64 编码后为	MA==
toString	Base64 编码后为	dG9TdHJpbmc=

浏览器中有自带的 Base64 编码和解码的函数，其中 btoa 用来编码，atob 用来解码。但在实际的混淆应用中，最好还是自己去实现它们，然后加以混淆。注意，字符串加密后，需要把对应的解密函数也放入代码中，才能正常运行。这里就用 atob 来代替 Base64 解码，处理后的代码为：

```
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {
    var \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
    var Week = ['\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94', '\u516d'];
    eval(String.fromCharCode(115, 116, 114, 32, 61, 32, 115, 116, ...));
    str = str[atob('cmVwbGFjZQ==')]/MM/, (this[atob('Z2V0TW9udGg=')]() + 1) > 9 ? (this
[atob('Z2V0TW9udGg=')]() + 1)[atob('dG9TdHJpbmc=')]() : atob('MA==') + (this[atob
('Z2V0TW9udGg=')]() + 1));
    str = str[atob('cmVwbGFjZQ==')]/dd|DD/, this[atob('Z2V0RGF0ZQ==')]() > 9 ? this[atob
('Z2V0RGF0ZQ==')]()[atob('dG9TdHJpbmc=')]() : atob('MA==') + this[atob('Z2V0RGF0ZQ==')]());
}
```

```
        return str;
    }
    console.log( new \u0077\u0069\u0065\u0064\u0066\u0077[ '\u0044\u0061\u0074\u0065' ] )(
    [String.fromCharCode(102, 111, 114, 109, 97, 116)]( '\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64' ) );
    //输出结果 2020-07-04
```

在实际混淆应用中,标识符必须处理成没有语义的,不然很容易就定位到关键代码。此外,建议减少使用系统自带的函数,自己去实现相应的函数。因为不管如何混淆,最终执行过程中,系统函数的名字是固定的,通过 Hook 极易定位到关键代码。如何完成 JS Hook 将在第 10 章中介绍。

根据写法不同,代码中有一些字符串常量没法加密和拼接。如以下代码:

```
var obj = { 'name': 'xiaojianbang' };
/*
    上面这种方式, name 虽然是一个字符串,但是不能拼接和加密。当然表示成十六进制或者
    Unicode 形式还是可以的
    var obj = { '\x65\x61\x6d\x65': 'xiaojianbang' };
    console.log( obj[ '\x65\x61\x6d\x65' ] ); // xiaojianbang
    console.log( obj.name ); // xiaojianbang
*/
var obj = {};
var str = 'na';
obj[str + 'me'] = 'xiaojianbang';
//用这种方式给对象增加属性,属性名可以加密和拼接
```

6.1.6 数值常量加密

算法加密过程中,会使用一些固定的数值常量,如 MD5 中的常量 0x67452301、0xefcdab89、0x98badcfe 和 0x10325476,以及 sha1 中的常量 0x67452301、0xefcdab89、0x98badcfe、0x10325476 和 0xc3d2e1f0。因此,在标准算法逆向中,会通过搜索这些数值常量,来定位代码关键位置,或者确定使用的是哪个算法。当然,在代码中不一定会写十六进制形式,如 0x67452301,在代码中可能会写成十进制的 1732584193。安全起见,可以把这些数值常量也进行简单加密。

可以利用位异或的特性来加密。例如,如果 $a \oplus b = c$,那么 $c \oplus b = a$ 。以 sha1 算法中的 0xc3d2e1f0 常量为例, $0xc3d2e1f0 \oplus 0x12345678 = 0xd1e6b788$,那么在代码中可以用 $0xd1e6b788 \oplus 0x12345678$ 来代替 0xc3d2e1f0,其中 0x12345678 可以理解成密钥,它可以随机生成。

混淆方案不一定是单一使用,各种方案之间可以结合使用。上述方法中两个数字进行位异或,实际上就是一个二项式。关于二项式的混淆,会在 6.2.3 节中详细介绍。

6.2 增加 JS 逆向者的工作量

在 6.1 节中介绍了一部分的混淆手段,现在应该对 JS 混淆有了认识。但实际上只是处理了一些常量,防护力度并不高。混淆的目的是为了增加破解的难度和时间。因此本节从这方面入手,继续介绍更深入的内容。

6.2.1 数组混淆

该类混淆没有统一称呼,数组混淆是本书中的叫法。将 6.1.5 节中的代码跟 6.1 节中的代码对比,在改变对象属性的访问方式后,产生了很多原本没有的字符串。虽然在前面的介绍中,已经对它们做了一系列的处理,但是遇到有混淆逆向经验的逆向开发者,破解这里的混淆十分容易。本节的方案是将所有的字符串都提取到一个数组中,然后在需要引用字符串的地方,全部都以数组下标的方式访问数组成员。例如:

```
var bigArr = ['Date', 'getTime', 'log'];
console[bigArr[2]]( new window[bigArr[0]]()[bigArr[1]]() );
//console.log( new window.Date().getTime() )
```

这里展示的代码,阅读难度已经大大增加。当代码为上千行,数组提取的字符串也有上千个。在代码中要引用字符串时,全都以 bigArr[1001]和 bigArr[1002]访问,就会大大增加理解难度,不容易建立对应关系。

在其他语言中,同一个数组只能存放同一种类型。但是 JS 语法灵活,同一个数组中,可以同时存放各种类型,如布尔值、字符串、数值、数组、对象和函数等。例如:

```
var bigArr = [
    true,
    'xiaojianbang',
    1000,
    [100,200,300],
    {name: 'xiaojianbang', money: 0},
    function(){console.log('Hello')}
];
console.log( bigArr[0] );           //true
console.log( bigArr[1] );           //xiaojianbang
console.log( bigArr[2] );           //1000
console.log( bigArr[3][0] );        //100
console.log( bigArr[4].money );     //0
console.log( bigArr[5]() );         //Hello
```

因此,可以把代码中的一部分函数提取到大数组中。如 6.1.5 节中的代码 String.fromCharCode 就可以提取到大数组中。为了安全,通常会对提取到数组中的字符串进行加密处理,把代码处理成字符串就可以进行加密了。对于这个函数,改写为以下形式:



视频讲解

```
console.log( ""['constructor']['fromCharCode'](120) ); //输出 x
```

最前面可以是任意的字符串对象,也可以是空字符串。constructor 代表获取构造函数,因此""['constructor']等同于 String。这样就全部变成字符串,可以进行字符串加密。处理后的代码如下:

```
var bigArr = [
    '\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94',
    '\u516d', 'cmVwbGFjZQ==', 'Z2V0TW9udGg=', 'dG9TdHJpbmc=',
    'Z2V0RGF0ZQ==', 'MA==', ""['constructor']['fromCharCode']
];
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {
    var \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
    var Week = [bigArr[0], bigArr[1], bigArr[2], bigArr[3], bigArr[4], bigArr[5], bigArr
[6]];
    eval(bigArr[12](115, 116, 114, 32, 61, 32, 115, 116, ... ));
    str = str[atob(bigArr[7])]/MM/, (this[atob(bigArr[8])]() + 1) > 9 ? (this[atob(bigArr
[8])]() + 1)[atob(bigArr[9])]() : atob(bigArr[11]) + (this[atob(bigArr[8])]() + 1));
    str = str[atob(bigArr[7])]/dd/DD/, this[atob(bigArr[10])]() > 9 ? this[atob(bigArr
[10])()][atob(bigArr[9])]() : atob(bigArr[11]) + this[atob(bigArr[10])]() );
    return str;
}
console.log( new \u0077\u0069\u006e\u0064\u006f\u0077[ '\u0044\u0061\u0074\u0065' ] ( )
[bigArr[12](102, 111, 114, 109, 97, 116)]( '\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64' ) );
//输出结果 2020-07-04
```

这段代码在不使用动态调试,也不使用 AST 的情况下,可读性很差。但是 JS 代码混淆仍将继续。

6.2.2 数组乱序

观察 6.2.1 节中处理后的代码,数组成员与被引用的地方是一一对应的。如引用 bigArr[12] 的地方,需要的是 String.fromCharCode 函数,而该数组中下标为 12 的成员,也是这个函数。将数组顺序打乱可以解决这个问题,不过在数组顺序混乱后,本身的代码也引用不到正确的数组成员。此处的解决方案是,在代码中内置一段还原顺序的代码。可以使用以下代码打乱数组顺序:

```
var bigArr = [
    '\u65e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94',
    '\u516d', 'cmVwbGFjZQ==', 'Z2V0TW9udGg=', 'dG9TdHJpbmc=',
    'Z2V0RGF0ZQ==', 'MA==', ""['constructor']['fromCharCode']
];
(function(arr, num){
    var shuffer = function(nums){
        while( -- nums){
```

```

        arr.unshift(arr.pop());
    }
};
shuffer(++num);
}(bigArr, 0x20));
console.log( bigArr );
//[ "cmVwbGFjZQ==", "Z2V0TW9udGg=", "dG9TdHJpbmc=", "Z2V0RGF0ZQ==", "MA==", f, "H", "
—", "二", "三", "四", "五", "六"]
//console.log( bigArr[5](120) );    //输出 x

```

在这段代码中,有一个自执行的匿名函数。实参部分传入的是数组和一个任意数值。在这个函数内部,通过对数组进行弹出和压入操作来打乱顺序。除此之外,只要控制台输出,Unicode 处理后的字符串就变成原来的中文。这就是之前说的十六进制字符串和 Unicode 都很容易被还原。

String.fromCharCode 函数被移动到了下标为 5 的地方,但代码处引用的仍是 bigArr [12],所以需要把还原数组顺序的函数放入代码中,还原数组顺序的代码逆向编写即可,如下所示:

```

var bigArr = [
    'cmVwbGFjZQ==', 'Z2V0TW9udGg=', 'dG9TdHJpbmc=', 'Z2V0RGF0ZQ==',
    'MA==', ""['constructor']['fromCharCode'], '\u65e5', '\u4e00',
    '\u4e8c', '\u4e09', '\u56db', '\u4e94', '\u516d'
];
(function(arr, num){
    var shuffer = function(nums){
        while( -- nums){
            arr['push'](arr['shift']());
        }
    };
    shuffer(++num);
})(bigArr, 0x20));
//后面的代码与 6.2.1 节中处理后的代码一致,这里省略

```

注意,还原数组顺序的函数里用到的字符串,不能再提取到 bigArr 中。

6.2.3 花指令

添加一些没有意义却可以混淆视听的代码,是花指令的核心。这里介绍一种比较简单的花指令实现方式,以 6.1 节的代码为例:

```

str = str.replace(/MM/, (this.getMonth() + 1) > 9 ? (this.getMonth() + 1).toString() : '0'
+ (this.getMonth() + 1));

```

把 this.getMonth()+1 这个二项式改为如下形式:

```
function _0x20ablfxe1(a, b){
    return a + b;
}
//_0x20ablfxe1(this.getMonth(), 1);
_0x20ablfxe1(new Date().getMonth(), 1);    //输出 7
//为了能够在控制台正常运行,把 this 改成 new Date()
```

本质是把二项式拆分成三部分：二项式的左边、二项式的右边和运算符。二项式的左边和右边作为另外一个函数的两个参数，二项式的运算符作为该函数的运行逻辑。这个函数本身是没有意义的，但它能瞬间增加代码量，从而增加JS逆向者的工作量。

二项式转变为函数时，进行多级嵌套，代码如下：

```
function _0x20ablfxe2(a, b){
    return a + b;
}
function _0x20ablfxe1(a, b){
    return _0x20ablfxe2(a, b);
}
_0x20ablfxe1(new Date().getMonth(), 1);    //输出 7
```

这个案例较为简单，但是在实际混淆中，代码可能有几千行，函数定义部分与调用部分往往相差甚远。在第11章的实战中，就会碰到类似的代码。另外，具有相同运算符的二项式，并不是一定要调用相同的函数。如把 `'0' + (this.getMonth() + 1)` 这个二项式改为如下所示代码：

```
function _0x20ablfxe2(a, b){
    return a + b;
}
function _0x20ablfxe1(a, b){
    return _0x20ablfxe2(a, b);
}
function _0x20ablfxe3(a, b){
    return a + b;
}
function _0x20ablfxe4(a, b){
    return _0x20ablfxe3(a, b);
}
_0x20ablfxe4('0', _0x20ablfxe1(new Date().getMonth(), 1));
//输出 "07"
```

上面介绍的是二项式转变为函数的花指令，其实函数调用表达式也可以处理成类似的花指令。代码如下：

```
function _0x20ablfxe2(a, b, c){
    return a(b, c);
}
```

```
function _0x20ab1fxel(a, b, c){
    return _0x20ab1fxe2(a, b, c);
}
str = _0x20ab1fxel(
    str.replace,
    /MM/,
    (this.getMonth() + 1) > 9 ? (this.getMonth() + 1).toString() : '0' + (this.getMonth() +
1));
```

花指令的生成方案,并不是只有这些。在 6.3.2 节中,还会演示另外一种插入花指令的方式。

6.2.4 jsfuck

jsfuck 也可以算是一种编码。它能把 JS 代码转化成只用 6 个字符就可以表示的代码，并可以正常执行。这 6 个字符分别是“(”、“+”、“!”、“[”、“.”和“)”。转换后的 JS 代码难以阅读，可作为简单的保密措施，如数值常量 8 转成 jsfuck 后为：

(! + [] + !! [] + !! [] + !! [] + !! [] + !! [] + !! [] + !! [] + !! [] + [])

接下来介绍 jsfuck 的基本原理，+ 是 JS 中的一个运算符，当它作为一元运算符使用时，代表强转为数值类型。[] 在 JS 中表示空数组，因此 +[] 等于 0，! +[] 等同于 !0。JS 是一种弱类型的语言，弱类型并不是代表没有类型，是指 JS 引擎会在适当的时候，自动完成类型的隐式转换。! 是 JS 中的取反，这时需要一个布尔值。在 JS 中，七种值为假值，其余均为真值。这七种值分别是 false、undefined、null、0、-0、NaN 和 ""。因此，0 转换为布尔值为 false，再取反就是 true，也就是 !+[] == true。又如 ![]，数组转换成布尔值为 true，然后两次取反，依旧等于 true。JS 中的 + 作为二元运算符时，假如有一边是字符串，就代表着拼接；两边都没有字符串，就代表着数值相加。true 转换为数值等于 1。剩余的部分原理相同，不再赘述。

在实际开发中,jsfuck 的应用有限,只会应用于 JS 文件中的一部分代码。主要原因是它的代码量非常庞大且还原它较为容易。例如,把上述代码直接输入控制台运行,就会输出 8。

一些网站之所以用它进行加密,是因为个别情况下,把整段 jsfuck 代码输入控制台运行会报错,尤其是当它跟别的代码混杂时。例如以下代码:

[illegible]


```
function test1(){  
    var a = 1000;  
    var b = a + 2000;  
    var c = b + 3000;  
    var d = c + 4000;  
    var e = d + 5000;  
    var f = e + 6000;  
    return f;  
}  
console.log( test1() );  
//输出 21000
```

混淆 test1 函数中的代码执行流程为：首先把代码分块，且打乱代码块的顺序，分别添加到不同的 case 块中。方便起见，就处理成一行代码对应一个 case 块的形式，代码如下：

```
switch(){  
    case '1':  
        var c = b + 3000;  
    case '2':  
        var e = d + 5000;  
    case '3':  
        var d = c + 4000;  
    case '4':  
        var f = e + 6000;  
    case '5':  
        var b = a + 2000;  
    case '6':  
        return f;  
    case '7':  
        var a = 1000;  
}
```

可以看出，当代码块打乱后，如果想要跟原先的执行顺序一样，那么 case 块的跳转顺序应该是 7、5、1、3、2、4、6。只有 case 块按照这个流程执行，才能跟原始代码的执行顺序保持一致。

其次，需要一个循环。因为 switch 语句只计算一次 switch 表达式，它的执行流程如下：

- (1) 计算一次 switch 表达式。
- (2) 把表达式的值与每个 case 的值进行对比(这里是 === 的匹配，不转换类型)。
- (3) 如果存在匹配，则执行对应 case 块。

因此，代码可以改成如下形式：

```
while (!![]) {  
    //思考 switch 中的表达式该怎么写  
    switch() {  
        case '1':  
            var c = b + 3000;  
    }
```

```
        continue;
        //每执行一次 case 块中的代码,就跳到循环末尾,继续下一次循环
    case '2':
        var e = d + 5000;
        continue;
    case '3':
        var d = c + 4000;
        continue;
    case '4':
        var f = e + 6000;
        continue;
    case '5':
        var b = a + 2000;
        continue;
    case '6':
        return f;
        continue;
    case '7':
        var a = 1000;
        continue;
    }
    break;
    //当 switch 计算出来的表达式的值与每个 case 的值都不匹配,代码就会运行到这里,再跳出
    循环
}
```

这是一个死循环,所以需要有一个边界条件来结束循环。假如函数有 return 语句,那么执行到对应的 case 块后,会直接返回。假如函数没有 return 语句,代码执行到最后,就需要让 switch 计算出来的表达式的值与每个 case 的值都不匹配,那么就会执行最后的 break 来跳出循环。

在这个案例里,return 语句后面的 continue 语句是会被执行的,但留着不影响代码运行。假如这是一段由 AST 自动处理出来的代码,这样做更具通用性,不需要考虑函数的最后一条语句是否是 return 语句。

最后,需要构造一个分发器,里面记录了代码执行的真实顺序。例如, var arrStr = '7|5|1|3|2|4|6'.split('|'), i = 0;,把这个字符串'7|5|1|3|2|4|6'通过 split 分割成一个数组。i 作为计数器,每次递增,按顺序引用数组中的每一个成员。因此,switch 中的表达式就可以写成 switch(arrStr[i++])。完整代码如下所示:

```
function test2(){
    var arrStr = '7|5|1|3|2|4|6'.split('|'), i = 0;
    while (!![]) {
        switch(arrStr[i++]){
            case '1':
                var c = b + 3000;
                continue;
            case '2':
```

```

        var e = d + 5000;
        continue;
    case '3':
        var d = c + 4000;
        continue;
    case '4':
        var f = e + 6000;
        continue;
    case '5':
        var b = a + 2000;
        continue;
    case '6':
        return f;
        continue;
    case '7':
        var a = 1000;
        continue;
    }
    break;
}
}
console.log( test2() );
//输出 21000

```

再来解释 `switch(arrStr[i++])` 的作用。`i` 的初始值为 0, 会先取到 `arrStr[0]`, 然后 `i` 增加 1, 再取到 `arrStr[1]`, 以此类推。假如函数有 `return` 语句, 执行到最后一个 `case` 块时, 函数返回, 循环也退出了。假如函数没有 `return` 语句, 当 `i` 一直递增到数组越界时, 就会取到 `undefined` (JS 中访问数组越界不会报错), 然后执行最后的 `break` 跳出循环。

在理解了简单的案例后, 就可以对 6.2.2 节中的代码做进一步混淆, 处理后的代码如下:

```

//最开始的大数组
var bigArr = [
    'cmVwbGFjZQ==', 'Z2V0TW9udGg=', 'dG9TdHJpbmc=', 'Z2V0RGF0ZQ==', 'MA==',
    '"[' 'constructor']["fromCharCode', '\u65e5', '\u4e00', '\u4e8c',
    '\u4e09', '\u56db', '\u4e94', '\u516d'
];
//还原数组顺序的自执行函数
(function(arr, num){
    var shuffer = function(nums){
        while(--nums){
            arr['push'](arr['shift']());
        }
    };
    shuffer(++num);
})(bigArr, 0x20));
//本小节处理的 switch 流程平坦化
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {

```

```

var arrStr = '7|5|1|3|2|4'.split('|'), i = 0;
while (!![]) {
    switch(arrStr[i++]){
        case '1':
            eval(String.fromCharCode(115, 116, 114, 32, 61, 32, 115, 116 ..... ));
            continue;
        case '2':
            str = str[atob(bigArr[7])](/dd/DD/, this[atob(bigArr[10])]() > 9 ? this[atob(bigArr[10])]() [atob(bigArr[9])]() : atob(bigArr[11]) + this[atob(bigArr[10])]());
            continue;
        case '3':
            str = str[atob(bigArr[7])](/MM/, (this[atob(bigArr[8])]() + 1) > 9 ? (this[atob(bigArr[8])]() + 1)[atob(bigArr[9])]() : atob(bigArr[11]) + (this[atob(bigArr[8])]() + 1));
            continue;
        case '4':
            return str;
            continue;
        case '5':
            var Week = [bigArr[0], bigArr[1], bigArr[2], bigArr[3], bigArr[4], bigArr[5], bigArr[6]];
            continue;
        case '7':
            var \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
            continue;
    }
    break;
}
}
console.log( new \u0077\u0069\u006e\u0064\u006f\u0077[ '\u0044\u0061\u0074\u0065' ] ( )
[bigArr[12](102, 111, 114, 109, 97, 116)]( '\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64' ) );
//输出结果 2020-07-04

```

JS语法比较灵活,case后面跟的值可以是字符/字符串,也可以是数值还可以是对象或者数组。

6.3.2 逗号表达式混淆

逗号运算符的主要作用是多个表达式或语句连接成一个复合语句。6.3.1节中的test1函数等价于：

```

function test1(){
    var a, b, c, d, e, f;
    return a = 1000,
        b = a + 2000,
        c = b + 3000,

```

```
d = c + 4000,  
e = d + 5000,  
f = e + 6000,  
f  
}  
console.log( test1() );  
//输出 21000
```

return 语句后通常只能跟一个表达式,它会返回这个表达式计算后的结果。但是逗号运算符可以把多个表达式连接成一个复合语句。因此上述代码中,return 语句的使用也是没有问题的,它会返回最后一个表达式计算后的结果,但是前面的表达式依然会执行。上述案例只是单纯的连接语句,没有混淆力度。下面再介绍一个案例,代码如下:

```
var a = (a = 1000, a += 2000);  
console.log( a );  
//输出 ???
```

第一行代码中,括号代表这是一个整体,也就是把(a=1000, a+=2000)整体赋值给 a 变量。这个整体返回的结果和 return 语句是一样的,会先执行 a=1000,然后执行 a+=2000,再把结果赋值给 a 变量,最终 a 变量的值为 3000。

明白了上述原理后,再介绍逗号运算符的混淆,以本节中的 test1 函数为例,处理思路如下。

- (1) 执行 a=1000,再执行 a+2000,代码可以改为(a=1000, a+2000)。
- (2) 接着赋值给 b,代码可以改为 b=(a=1000, a+2000)。
- (3) 执行 b+3000,代码可以改为(b=(a=1000, a+2000), b+3000)。
- (4) 接着赋值给 c,代码可以改为 c=(b=(a=1000, a+2000), b+3000)。
- (5) 执行 c+4000,代码可以改为(c=(b=(a=1000, a+2000), b+3000), c+4000)。
- (6) 以此类推。

处理后的代码为:

```
function test2(){  
    var a, b, c, d, e, f;  
    return f = (e = (d = (c = (b = (a = 1000, a + 2000), b + 3000), c + 4000), d +  
5000), e + 6000);  
}  
console.log( test2() );  
//输出 21000
```

这段代码有一个声明一系列变量的语句。这个语句很多余,可以放到参数列表上,这样就不需要 var 声明了。另外,既然逗号运算符连接多个表达式,只会返回最后一个表达式计算后的结果,那么可以在最后一个表达式之前插入不影响结果的花指令。最终处理后的代码如下:

```
function test2(a, b, c, d, e, f){
    return f = (e = (d = (c = (b = (a = 1000, a + 50, b + 60, c + 70, a + 2000), d + 80,
    b + 3000), e + 90, c + 4000), f + 100, d + 5000), e + 6000);
}
console.log( test2() );
// 输出 21000
```

上述代码中 $a+50$ 、 $b+60$ 、 $c+70$ 、 $d+80$ 、 $e+90$ 、 $f+100$ 这些花指令并无实际意义,不影响原先的代码逻辑。test2 虽有 6 个参数,但是不传参也可以调用,只不过各参数的初始值为 undefined。

逗号表达式混淆不仅能处理赋值表达式,还能处理调用表达式、成员表达式等。考虑下面这个案例:

```
var obj = {
    name: 'xiaojianbang',
    add: function(a, b){
        return a + b;
    }
}
function sub(a, b){
    return a - b;
}
function test(){
    var a = 1000;
    var b = sub(a,3000) + 1;
    var c = b + obj.add(b, 2000);
    return c + obj.name;
}
```

test 函数中有函数调用表达式 sub,还有成员表达式 obj.add 等,可以使用以下两种方法对其进行处理。

(1) 提升变量声明到参数中。

(2) $b=(a=1000,sub)(a,3000)+1$ 中的 $(a=1000,sub)$ 可以整体返回 sub 函数,然后直接调用,计算的结果加 1 后赋值给 b(等号的运算符优先级很低)。同理,如果 sub 函数改为 obj.add 的话,可以处理成 $(a=1000,obj.add)(a,3000)$ 或者 $(a=1000,obj).add(a,3000)$ 。

第 2 种方法是调用表达式在等号右边的情况。例如 test 函数中的第 3 条语句里面的 $b+obj.add(b,2000)$,可以对 obj.add 进行包装,处理成 $b+(0,obj.add)(b,2000)$ 或者 $b+(0,obj).add(b,2000)$,括号中的 0 可以是其他花指令。

综上所述,上述案例中的代码,可以处理成如下形式:

```
var obj = {
    name: 'xiaojianbang',
    add: function(a, b){
```

```

        return a + b;
    }
}
function sub(a, b){
    return a - b;
}
function test() {
    return c = (b = {a = 1000, sub}(a, 3000) + 1, b + (0, obj).add(b, 2000)),
        c + (0, obj).name;
}

```

在理解了简单的案例后,就可以对 6.2.2 节中的代码做进一步混淆,处理后的代码如下:

```

//最开始的大数组
var bigArr = [
    'cmVwbGFjZQ==', 'Z2V0TW9udGg=', 'dG9TdHJpbmc=', 'Z2V0RGF0ZQ==', 'MA==',
    '"['constructor'] ['fromCharCode', '\u65e5', '\u4e00', '\u4e8c',
    '\u4e09', '\u56db', '\u4e94', '\u516d'
];
//还原数组顺序的自执行函数
(function(arr, num){
    var shuffer = function(nums){
        while( --nums){
            arr['push'](arr['shift']());
        }
    };
    shuffer(++num);
})(bigArr,0x20));
//本节处理的代码
//把原先的变量定义提取到参数列表中
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr, str, Week) {
    //因为基本上都会处理成一行代码,所以 return 语句可以提到最上面
    return str =
        (str = (
            Week = (
                \u0073\u0074\u0072 = \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074
                \u0072,
                [bigArr[0], bigArr[1], bigArr[2], bigArr[3], bigArr[4], bigArr[5], bigArr
                [6]]
                //上面这个表达式的结果,会赋值给 Week
            ),
            eval(bigArr[12](115, 116, 114, 32, 61, 32, 115, 116, ...)),
            str[atob(bigArr[7])](/MM/, (this[atob(bigArr[8])]() + 1) > 9 ? (this[atob(bigArr
            [8])]() + 1)[atob(bigArr[9])](): atob(bigArr[11]) + (this[atob(bigArr[8])]() + 1))
            //上面这个表达式的结果,会赋值给第二个 str
        ),
            str[atob(bigArr[7])](/dd DD/, this[atob(bigArr[10])]() > 9 ? this[atob(bigArr
            [10])]()[atob(bigArr[9])](): atob(bigArr[11]) + this[atob(bigArr[10])]())

```



```
//上面这个表达式的结果,会赋值给第一个 str
);
}
console.log( new \u0077\u0069\u0064\u0066\u0077[ '\u0044\u0061\u0074\u0065' ] ()
[bigArr[12][102, 111, 114, 109, 97, 116)]('\x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64' ) );
//输出结果 2020-07-04
```

最后介绍逗号表达式混淆的还原技巧。在逗号表达式混淆中,通常需要使用括号来分组。定位到最里面的那个括号,一般就是第一条语句。然后从里到外,一层层地根据括号对应关系,还原语句顺序。如果用 AST 还原逗号表达式混淆,就不用这么麻烦地找对应关系,几行代码就可以解决问题。在后续章节中会有详细介绍。

6.4 其他代码防护方案

6.4.1 eval 加密

在前文中已经接触过 eval 函数了,本节将介绍 eval 加密。此处以 6.1 节的代码为例说明 eval 加密,加密的代码格式化后如下所示:

```
eval(function(p,a,c,k,e,r){
    e = function(c) {
        return c.toString(36)
    };
    if ('0'.replace(0, e) == 0) {
        while (c--)
            r[e(c)] = k[c];
        k = [function(e) {
            return r[e] || e
        }];
        e = function() {
            return '[2-8a-f]'
        };
        c = 1
    };
    while (c--)
        if (k[c])
            p = p.replace(new RegExp('\\b' + e(c) + '\\b', 'g'), k[c]);
    return p
})('7.prototype.8 = function(a){b 2 = a;b Week = [\'日\',\'一\',\'二\',\'三\',\'四\',\'五\',\'六\',\'日\'];2 = 2.4(/c|YYYY/,3.getFullYear());2 = 2.4(/d/, (3.5() + 1)> 9? (3.5() + 1).e():\'0\' + (3.5() + 1));2 = 2.4(/f|DD/, 3.6()> 9? 3.6().e():\'0\' + 3.6());return 2};console.log(new 7().8(\'c - d - f\'));', [], 16, '|str|this|replace|getMonth|getDate|Date|format|formatStr|var|yyyy|MM|toString|dd'.split('|'), 0, {}))
```



视频讲解

这段代码的一个 `eval()` 函数,它用来把一段字符串当作 JS 代码来执行。也就是说,传给 `eval()` 的参数是一段字符串。但在上述代码中,传给 `eval()` 函数的参数是一个自执行的匿名函数。这说明,这个匿名函数执行后会返回一段字符串,并且用 `eval()` 执行这段字符串,执行效果与 `eval` 加密前的代码效果等同。那就可以把这个匿名函数理解成是一个解密函数了。由此可见,`eval` 加密其实和 `eval()` 关系不大,`eval()` 只是用来执行解密出来的代码。

再来观察传给这个匿名函数的实参部分。观察第 1 个实参 `p` 和第 4 个实参 `k`。可以看出处理方式很简单,提取原始代码中的一部分标识符,然后用它自己的符号占位,最后再对应替换回去就解密了。

最后介绍 `eval` 解密。这个比较容易,既然这个自执行的匿名函数就是解密函数,把上述代码中的 `eval` 删去,剩余代码在控制台中执行,就得到原始代码。

6.4.2 内存爆破

内存爆破是在代码中加入死代码,正常情况下这段代码不执行,当检测到函数被格式化或者函数被 `Hook`,就跳转到这段代码并执行,直到内存溢出,浏览器会提示 `Out of Memory` 程序崩溃。内存爆破的代码如下所示:

```
var d = [0x1, 0x0, 0x0];
function b(){
    for(var i = 0x0, c = d.length; i < c; i++){
        d.push(Math.round(Math.random()));
        c = d.length;
    }
}
```

这段代码中的 `for` 循环是一个死循环,但它的形式不像 `while(true)` 这样明显。尤其是代码混淆以后,更具迷惑性。这段代码其实是从以下这段代码简化而来:

```
this['NsTJKl'] = [0x1, 0x0, 0x0];
...
_0x4b1809['prototype']['xTDWoN'] = function (_0x597ca7) {
    for (var _0x3e27c4 = 0x0, _0x192434 = this['NsTJKl']['length']; _0x3e27c4 < _0x192434; _0x3e27c4++) {
        this['NsTJKl']['push'](Math['round'](Math['random']()));
        _0x192434 = this['NsTJKl']['length'];
    }
    return _0x597ca7(this['NsTJKl'][0x0]);
}
```

for 循环的结束条件是 `_0x3e27c4 < _0x192434`,其中 `_0x192434` 的初始化值是数组的大小。看着像是一个遍历数组的操作,但是在循环中,又往数组中 `push` 了成员,接着又

重新给 `_0x192434` 赋值为数组的大小。这时这段代码就永远也不会结束了,直到内存溢出。

6.4.3 检测代码是否格式化

检测的思路很简单,在 JS 中,函数是可以转为字符串的。因此可以选择一个函数转为字符串,然后跟内置的字符串对比或者用正则匹配。函数转为字符串很简单,代码如下:

```
function add(a, b){return a + b;}
console.log( add + '');
console.log( add.toString() );
// "function add(a, b){return a + b;}"
```

在 Chrome 开发者工具中,把代码格式化后,会产生一个后缀为 `formatted` 的文件。之后这个文件中设置断点,触发断点后,会停在这个文件中。但是,这时把某个函数转为字符串,取到的依然是格式化之前的代码,如图 6-1 所示。



图 6-1 在 Chrome 中格式化代码

上述检测方法检测不到这种情况。那么,上述检测方法的应用场景是什么?在算法逆向中,分析完算法,为了得到想要的结果,就需要实现这个算法。简单的算法一般可以直接调用现成的加密库。复杂的算法就会选择直接修改原文件,然后运行得到结果。把格式化后的代码保存成一个本地文件,这时某个函数转为字符串,取到的就是格式化后的结果了,如图 6-2 所示。

是否触发格式化检测,关键是看原文件中是否有格式化。接着把 6.4.2 节中的内存爆破代码加入其中。检测到格式化就跳转到内存爆破代码中执行,程序会崩溃。

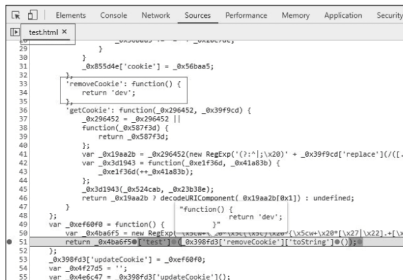


图 6-2 格式化代码后的结果

6.5 小结

混淆的目的是增加逆向开发者的工作量。例如，原本一小时就解决的算法，混淆后可能需要几天才能解决。当算法每天更新，逆向开发者自然就放弃了。目前市面上已有此类方案，只不过变化的算法仅限于微调，如算法中的常量、算法加密前的参数顺序等。如果要实现此类方案，需要一种自动化处理代码的方案，AST 为此而生。

6.6 习题

1. 尝试把 'Hello,AST', 转为十六进制字符串。
2. `!+[]+!![]+!![]+!![]+[]` 计算后的结果是多少？
3. 以下代码执行后，输出的值是多少？

```
var o = {
  x: function (a) {
    return a + 1000;
  }
};
function a(z) {
  z = 2000;
  return function (x, s, t) {
    return (t = (s = (x = o.x(100), o)).x(200), t + z);
  })();
}
console.log( a() );
```

4. 将第 3 题中的逗号表达式混淆，手动还原成更易读的形式。
5. 将第 4 题中还原后的代码，手动改成 switch 流程平坦化的形式。



AST抽象语法树的原理与实现

学习了前面的章节,对于反爬虫概念以及混淆JS的还原有了大致了解。从本章开始,将逐步介绍AST相关原理,并使用Python实现简易JS解析器,将JS编译为AST抽象语法树。

7.1 理解AST抽象语法树

7.1.1 AST基本概念

在计算机科学中,抽象语法树(Abstract Syntax Code,AST)是源代码语法结构的一种抽象表示。它本质上是一棵多叉树,树上的每个节点都表示源代码中的一种结构。

AST之所以被称为抽象语法树,很大程度源于其语法结构——树状结构。在树状结构的语法表示中,原本联系紧密、结构紧凑的代码被切分成了不可再分的零碎词块,且语法不会表示出真实语法中的所有细节,由此带来了“抽象”的感觉。例如,编写一个简单的判断表达式 $1+1=2$,表达式在解析后得到的AST抽象语法树如图7-1所示。

7.1.2 AST在编译中的位置

回顾编译原理,使用任何编程语言都需要一些软件来处理源代码以便让计算机能够理解。该软件可以是解释器,也可以是编译器。无论使用的是解释型语言(JS、Python、Ruby),还是编译型语言(C#、Java、Rust),它们都有一个共同的部分:将源代码作为纯文本解析为AST抽象语法树的数据结构。

编译器分为五个部分:词法分析、语法分析、语义分析、中间代码的生成及优化和目标代码的生成。目标代码往往是以二进制的形式存在;解释器更像是一个“计算器”,输入的

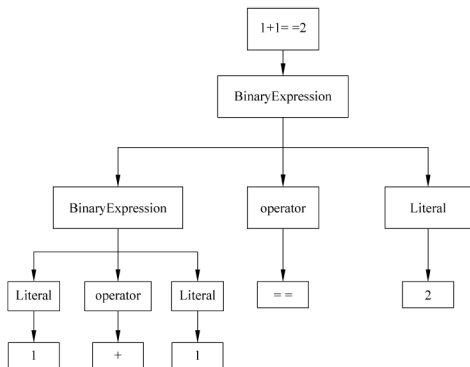


图 7-1 AST 抽象语法树

表达式会直接产出执行结果。二者在语法分析后都会生成 AST。如图 7-2 所示,从流程出发,不论是编译器还是解释器,由 JS 源代码到 AST 抽象语法树所需要的过程主要分为两步:将源代码进行词法分析,生成 token 符号流;token 符号流通过语法分析,生成语法树。

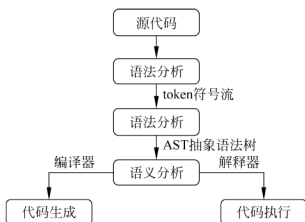


图 7-2 AST 的位置

7.1.3 AST 程序开发

AST 不仅用于编程语言的解释器和编译器开发。在计算机世界中,它还有多种应用。使用它最常见的方法之一是进行静态代码分析。静态分析器不执行输入的代码,但仍然需要理解代码的结构。例如,开发者想要实现一个程序,使用该程序可以将输入代码中的重复结构进行重构,或者将其中新的语法结构转化为向下兼容的代码。

实现这样的工具,可以不用自己编写解释器。目前 JS 拥有强大的 Babel 编译器可以帮助完成复杂任务,生成语法树输出,即 ASTs,它被广泛应用于代码转换。例如,希望实现一个将 JS 代码进行语法混淆的工具,在不改变代码的基本结构的情况下,把其中的变量和代码逻辑隐藏起来。或者实现一个简单的编译器,把输入的 Python 代码转化为 JS 代码。

事实上,ASTs 只是部分语言的不同表示方法。解析之前,它被表示为遵循一些规则的文本,这些规则构成了一种语言。解析之后,它被表示为一个树状结构,其中包含与输入文本完全相同的信息。因此,也可以进行反向解析,然后回到文本。

7.2 词法分析

7.2.1 词法分析基本原理

词法分析是将源代码转化为 AST 抽象语法树的第一步,是编译的基础。主要作用是对源代码进行从左到右的扫描,按照定义的词法规则识别单词,并且生成对应的单元供语法分析调用。完成词法分析任务的程序叫词法分析器。以具体的 JS 代码为例,以下是一个简单的赋值语句:

```
let x = y + 1 ;
```

词法分析器处理到这行源代码时,首先进行预处理,从左到右一个字符一个字符地读入,在字符逻辑上是一个单元的字符将会被组合起来,合成之后再对单元进行分类。每个类别会有一个名称方便识别,也可能会有数字类别,用于数理逻辑划分。在这一行代码中,预先设定如表 7-1 所示的词法规则。

表 7-1 词法规则表

数字类型	单 词 类 型	语 素	名 称
0	关键词	let	LET
1	赋值操作符	=	ASSIGN_SIGN
2	变量	x	IDENTIFIER
3	加法操作符	+	PLUS_SIGN
4	数字	1	INTEGER
5	语句结束	;	SEMICOLON

在词法分析器识别源代码时,计算机不知道这是以空格分隔的 7 个单词,只知道这是普通的 9 个字符构成的字符串。这里使用空格作为分隔符,将语素从输入字符串中分隔出来:

```
'let' 'x' '=' 'y' '+' '1' ';' ;
```

语素只是一类字符构成的单元,纯粹的字符单元切分是没有意义的。因而词法分析器还会为每个字符单元添加属性,根据定义的词法规则生成对应的属性词块,还可以带上字符单元对应的行号,这样方便输出错误信息和开发调试器,如下所示:

```
LET(0, 'let', 1)
IDENTIFIER(1, 'x', 1)
ASSIGN_SIGN(2, '=', 1)
IDENTIFIER(1, 'y', 1)
PLUS_SIGN(3, '+', 1)
INTEGER(4, '1', 1)
SEMICOLON(5, ';', 1)
```

每个词块都会包含字符单元本身,并将数字序号写入。这样的词块划分只是为了方便

用户识别,从机器角度出发,这段代码在数理逻辑上可以用数字 0121345 来代替。将输入代码分隔为单词、进而将单词进行分类的过程叫作单词化,生成的单词被用来进行语法分析。

7.2.2 Python 编写词法分析器

使用 Python 语言编写一个简单的词法分析器,以数字赋值表达式为例,将上述词块定义为 Token 类。每个 Token 都包含着类型、单词和行号,如下所示:

```
class Token:
    def __init__(self, type, literal, lineNumber):
        self.type = type
        self.literal = literal
        self.lineNumber = lineNumber

    @property
    def getType(self):
        return self.type

    @property
    def getLiteral(self):
        return self.literal

    @property
    def getLineNumber(self):
        return self.lineNumber

    def __str__(self):
        return "Type:{} ;Literal:{} ;LineNumber:{}".format(self.type, self.literal, self.lineNumber)
```

接着定义一个 Lexer 类充当词法分析器,把输入的源代码解析成 Token 词块。这里选择简单的解析方法,将字符一个一个地读入,判断读取的字符是否是特殊符号,如果有赋值符号或者加号等,就直接生成 Token 对象;如果没有,就将字符组合成字符串,直到遇到空格或者回车,然后判断生成的字符串对应词法分析表中的哪个分类,按照类别生成 Token 对象。首先定义词法分析器,代码如下:

```
class Lexer:
    def __init__(self, code):
        self.code = code
        self.position = 0
        self.readPosition = 0
        self.lineCount = 0
        self.char = ''
        self.initTokenType()

    def initTokenType(self):
        self.ILLEGAL = -2
        self.EOF = -1
        self.LET = 0
```



```
self.IDENTIFIER = 1
self.ASSIGN_SIGN = 2
self.PLUS_SIGN = 3
self.INTEGER = 4
self.SEMICOLON = 5
```

定义的 `initTokenType` 方法相当于词法分析表的分类,为不同的字符序列标记了不同的数字符号。接着实现字符的读入方法,作用是将输入源代码中的字符串一个字符一个字符地读入,在读取位置到达源代码末尾时终止。然后定义一个 `skip` 方法,用于跳过读取字符中的空格符号或者换行和制表符,如果遇到回车符或者换行符,会将行号加 1,如下所示:

```
def readChar(self):
    if self.readPosition >= len(self.code):
        self.char = 0
    else:
        self.char = self.code[self.readPosition]

    self.position = self.readPosition
    self.readPosition += 1

def skip(self):
    while (self.char == ' ' or self.char == '\t' or self.char == '\n'):
        if (self.char == '\t' or self.char == '\n'):
            self.lineCount += 1
        self.readChar()
```

遇到和词法规则匹配的字符时,需要根据其特定含义进行分类,而且在遇到多个字符有关联的情况下,会对字符单元进行组合,实现代码如下:

```
def nextToken(self):
    token = ''
    lineCount = self.lineCount
    self.skip()

    if self.char == '+':
        token = Token(self.PLUS_SIGN, '+', lineCount)
    elif self.char == '=':
        token = Token(self.ASSIGN_SIGN, '=', lineCount)
    elif self.char == ';':
        token = Token(self.SEMICOLON, ';', lineCount)
    elif self.char == 0:
        token = Token(self.EOF, '', lineCount)
    else:
        longstr = self.readIdentifier()
        if longstr:
            token = Token(self.IDENTIFIER, longstr, lineCount)
        else:
```

```
        longstr = self.readInteger()
        if longstr:
            token = Token(self.INTEGER, longstr, lineCount)

        if longstr == 'let':
            token = Token(self.LET, longstr, lineCount)

    self.readChar()
    return token
```

执行 nextToken 方法时,先跳过读入字符中的空格或换行符。如果遇到特殊符号,就会及时生成对应的 Token 类;如果遇到的是英文字母或者阿拉伯数字,就会判断组合情况,当读入的连续字符是有关联的,词法分析器会进行字符拼接,只有读取完整的字符单元才会停止。例如在读取以下代码时:

```
let test = 123 ;
```

let、test 和 123 是三个词法单元,需要将它们整体读取。在词法分析器遇到 test 时,先会读入 t、e、s 和 t 四个字符,然后通过 readIdentifier 方法将它们拼接为字符串 test,生成对应的字母 Token 类;在遇到数字 123 时,先读入 1、2 和 3 这三个字符,再经过 readInteger 方法拼接为字符串 123,生成相应的数字 Token 类。在 readIdentifier 方法和 readInteger 方法的编写之前,还需要分辨字母和数字两种数据类型,在 Python 中使用内置函数即可,如:

```
def isLetter(self, char):
    return char.isalpha()

def isDigit(self, char):
    return char.isdigit()

def readIdentifier(self):
    identifier = ''
    while self.isLetter(self.char):
        identifier += self.char
        self.readChar()

    if len(identifier) > 0:
        return identifier
    return False

def readInteger(self):
    integer = ''
    while self.isDigit(self.char):
        integer += self.char
        self.readChar()

    if len(integer) > 0:
        return integer
    return False
```

至此,简单的词法分析器编写完毕,接下来只需要写一个启动函数即可,如:

```
def startLex(self):
    self.readChar()

    tokens = []
    token = self.nextToken()
    while token.getType() != self.EOF:
        print('token:', token)
        tokens.append(token)
        token = self.nextToken()
    return tokens
```

最后,编写一段简单的测试代码,放入词法分析器中进行词法分析,如下所示:

```
if __name__ == '__main__':
    code = """let x = 1 ;
        let test = 123 ;
        """

    Lex = Lexer(code)
    Lex.startLex()
```

运行代码后,输出每次 Token 分词结果如下:

```
token: type:0 literal:let lineNumber:0
token: type:1 literal:x lineNumber:0
token: type:2 literal:= lineNumber:0
token: type:4 literal:1 lineNumber:0
token: type:5 literal;; lineNumber:0
token: type:0 literal:let lineNumber:0
token: type:1 literal:test lineNumber:1
token: type:2 literal:= lineNumber:1
token: type:4 literal:123 lineNumber:1
token: type:5 literal;; lineNumber:1
```

其中 let 对应的数字类型是 0,也就是词法规则表中的 LET 类型; x 和 test 对应数字类型 2,是词法规则表中的 IDENTIFIER 类型; 1 和 123 对应的数字类型是 4,是词法规则表中的 INTEGER 类型; 分号对应的数字类型是 5,是词法规则表中的 SEMICOLON 类型; 等号对应的数字类型是 1,是词法规则表中的 ASSIGN_SIGN 类型。此外,各词法单元行号也都是正确的,说明词法分析器运行无误。

7.3 语法分析

7.3.1 语法分析基本原理

语法分析是编译原理的核心部分。作用是接收词法分析给出的词块,在此基础上将词块组成各类的语法短语,分析是否是给定语法的正确句子,判断源代码在结构上是否正

确。目前常用的是自顶向下文法和自底向上文法。

1. 自顶向下文法

自顶向下文法是对面向目标的分析方法,从文法的开始符号出发,目的是推导出与输入的源程序串完全匹配的句子。如果输入的源程序串符合定义好的文法,就能推导出具体结果,反之则会报错。自顶向下文法的确定分析需要对文法进行限制,但实现起来简单方便,所以目前较为常用。自顶向下文法的不确定分析是带回溯的分析方法,本质上是一种穷举的试探方法,因效率低下,较少使用。

2. 自底向上文法

自底向上文法是对输入源程序从左向右扫描,并将得到的字符压入一个后进先出的栈数据结构中。在压入的同时进行分析,如果栈顶符号串形成某个句型的句柄或可归约串时,就用该产生式的左部非终结符代替相应右部的文法符号串,这称为一步归约。重复这一过程直到归约到栈中只剩文法的开始符号时则为分析成功,确认输入串是文法的句子。

在 7.2 节中介绍了词法分析器的开发,它对输入源代码进行了识别分类,把不同类别的单元划分到了不同的 Token 类中。例如字母构成的字符串,如果不是定义好的关键字,会被划分到 IDENTIFIER 类,而数字构成的字符串会被划分到 INTEGER 类中。

用以下语句为例:

```
let test = 1 ;
```

经过词法分析器的切分,会转化为以下形式:

```
LET IDENTIFIER ASSIGN_SIGN INTEGER SEMICOLON
```

接下来需要用语法分析器进行处理。语法分析器会判断输入的分类组合是否合法,如果组合是正确的,语法分析器会根据组合所形成的内在逻辑关系,构造出 AST 抽象语法树。根据这棵多叉树的数据结构,编辑器就可以为代码生成二进制指令,或者用解释器进行执行。

为了进行语法判断,需要严格定义每一行代码背后的逻辑结构。这里规定,let 关键字后必须跟一个字符串变量,且字符串变量后必须是数字的赋值语句,它的语法表达式如下所示:

```
LetStatement : = LET IDENTIFIER ASSIGN_SIGN INTEGER SEMICOLON
```

根据 LetStatement,可以判断输入的字符组合是否符合语法规则,对于以下代码:

```
let test 123 ;
```

在词法分析后得到的分类组合为:

```
LET IDENTIFIER INTEGER SEMICOLON
```

显然与已经定义好的语法规则不符。这时,语法分析器会分析出代码中的错误组合,并

对开发者报告错误。

此处 `LetStatement` 定义的规则只是为了说明。事实上, `INTEGER` 可以更改为 `EXPRESSION`, `EXPRESSION` 可以用来表示变量字符串、数字或者复杂的算术表达式。既然 `EXPRESSION` 可以包含复杂表达式,那就暗含了递归思想,需要再次深入到 `EXPRESSION` 里进行语法规则的匹配。

在接下来的语法规则的编写中,选择较为简单的自顶向下文法。

7.3.2 Python 编写语法分析器

本节的目标是开发一个可以解析以下代码的语法分析器:

```
let test = 123 ;
```

因为要构建 AST 抽象语法树,所以需要定义叶子节点 `Node`, `Statement` 代表着一句完整的代码, `Expression` 表示赋值表达式,它们都建立在 `Node` 节点的基础上,如下所示:

```
class Node:
    def __init__(self, props):
        self.tokenLiteral = ''

    def getLiteral(self):
        return self.tokenLiteral

class Statement(Node):
    def statementNode(self):
        return self

class Expression(Node):
    def __init__(self, props):
        super(Expression, self).__init__(props)
        self.tokenLiteral = props.token.getLiteral()

    def expressionNode(self):
        return self
```

在处理表达式的时候,变量定义为字符串 `Identifier` 类型,如下所示:

```
class Identifier(Expression):
    def __init__(self, props):
        super(Identifier, self).__init__(props)
        self.tokenLiteral = props.token.Literal
        self.token = props.token
        self.value = ''
```

目标是要解析以下语句:

```
let test = 123 ;
```

这行代码对应的语法表达式为：

```
LET IDENTIFIER ASSIGN_SIGN INTEGER SEMICOLON
```

在给出的代码中,let 是关键字,根据语法规则,后边跟随字母分类 IDENTIFIER,所以使用 Identifier 类解析时,props.token 对应类别是 IDENTIFIER 词块,token.getLiteral 方法得到的是字符串 test。等号后是一个数字,它的类别是 INTEGER,使用 Expression 类解析它时,得到的 token 类别是 INTEGER,所以 getLiteral 方法得到的是数组字符串 123。

接着定义一个 LetStatement 类来解析 let 语句。它由两部分组成:let 后的字符串变量;赋值表达式之后的数字、变量或者算术表达式,如下所示:

```
class LetStatement(Statement):
    def __init__(self, props):
        super(LetStatement, self).__init__(props)
        self.token = props.token
        self.name = props.identifier
        self.value = props.expression

        res = {'LetStatement': {}}
        res['LetStatement']['left'] = props.identifier.\
            getLiteral()
        res['LetStatement']['right'] = self.value.getLiteral()
        self.tokenLiteral = res
```

其中,props.token 对应关键字 let,props.identifier 对应关键字后的字符串变量,也就是 Identifier 类,props.expression 对应等号后的赋值语句,在这里暂时是一个数字,也就是 INTEGER。

然后定义整个输入代码文件的 Program 类,代码如下:

```
class Program:
    def __init__(self):
        self.statements = []

    def getLiteral(self):
        if len(self.statements) > 0:
            return self.statements[0].tokenLiteral()
        return ''
```

它由一系列 Statement 组成,一个 Statement 可以理解为一行以分号结束的代码。整个输入代码是以分号结束的 Statement 的集合,也有不以分号结尾的 Statement,例如条件判断:

```
if (...) {...}  
else {...}
```

接着构建语法规析的主体代码。在构造语法分析器时,需要传入词法分析器,因为语法分析器以词法分析的结果为输入。在初始化阶段,会先调用词法分析器的启动函数,对输入的源代码进行词法分析,词法分析得到一系列 token 字符单元作为语法分析器的输入。再声明两个指针,一个用于指向词法分析后得到 token 数组的当前字符单元,另一个指向下一个字符单元,如下所示:

```
class CompilerParser:  
    def __init__(self,lexer):  
        self.lexer = lexer  
        self.lexer.startLex()  
        self.tokenPosition = 0  
        self.currentToken = None  
        self.peakToken = None  
        self.nextToken()  
        self.nextToken()  
        self.program = Program()
```

在初始化中连续两次读入了 token,只有读入两个字符单元才能了解当前解析的代码的含义,nextToken 方法如下所示:

```
def nextToken(self):  
    self.currentToken = self.peakToken  
    if self.tokenPosition < len(self.lexer.tokens):  
        self.peakToken = self.lexer.tokens[self.tokenPosition]  
    self.tokenPosition += 1
```

例如输入的代码是 '1;',currentToken 对应的是 1,peakToken 对应的是分号,解析的时候就知道当前的代码表示一个整数 1。在初始化函数中,运行完这两行代码后,currentToken 会指向 token 数组中的第一个 token,peakToken 会指向第二个 token。

在 parseProgram 方法被调用时,语法分词器就正式开始运作了。首先读入一个完整的 Statement,这里的 Statement 就是以 let 关键字开头的语句,如:

```
def parseProgram(self):  
    while self.currentToken.type!= self.lexer.EOF:  
        state = self.parseStatement()  
        if state!= None:  
            self.program.statements.append(state)  
        self.nextToken()  
    return self.program  
  
def parseStatement(self):  
    if self.currentToken.Type== self.lexer.LET:  
        return self.parseLetStatement()
```

```
        else:
            return None

    def parseLetStatement(self):
        props = {}
        props['token'] = self.currentToken
        if (self.expectPeek(self.lexer.IDENTIFIER) == False):
            return None
        identProps = {}
        identProps['token'] = self.currentToken
        identProps['value'] = self.currentToken.Literal
        props['identifier'] = Identifier(identProps)

        if (self.expectPeek(self.lexer.ASSIGN_SIGN) == False):
            return None
        if (self.expectPeek(self.lexer.INTEGER) == False):
            return None

        exprProps = {}
        exprProps['token'] = self.currentToken
        props['expression'] = Expression(exprProps)

        if (self.expectPeek(self.lexer.SEMICOLON) == False):
            return None

        letStatement = LetStatement(props)
        return letStatement

    def curTokenIs(self, tokenType):
        return self.currentToken.Type == tokenType

    def peekTokenIs(self, tokenType):
        return self.peekToken.Type == tokenType

    def expectPeek(self, tokenType):
        if self.peekTokenIs(tokenType):
            self.nextToken()
            return True
        else:
            return False
```

在 `parseProgram` 方法中,每次会读入一个 `token`,如果读入的 `token` 不代表代码结束,就会使用 `parseStatement` 方法解析一条语句。

在 `parseStatement` 方法中,面对不同的关键词,会进行不同的方法调用。暂时只实现了 `let` 关键字的语句解析,所以判断输入的代码是否符合 `LetStatement` 的语法结构,进行 `let` 关键字判断的是 `parseLetStatement` 方法。它首先会根据语法结构判断,查看关键字 `let` 之后是否跟着字符串变量,如:


```
if (self.expectPeek(self.lexer.IDENTIFIER) == False):  
    return None
```

接着判断字符串变量后是否跟着赋值符号：

```
if (self.expectPeek(self.lexer.ASSIGN_SIGN) == False):  
    return None
```

在 let 语句的赋值符号后，目前规定只能跟数字类型。如果是数字类型，就构造 Expression 类，将数字封装进去，代码如下：

```
if (self.expectPeek(self.lexer.INTEGER) == False):  
    return None
```

```
exprProps = {}  
exprProps['token'] = self.currentToken  
props['expression'] = Expression(exprProps)
```

最后，let 语句会规定代码必须以分号结尾：

```
if (self.expectPeek(self.lexer.SEMICOLON) == False):  
    return None
```

接下来，只需要再添加一个入口方法：

```
def startParser(self):  
    self.parseProgram()  
    for i in self.program.statements:  
        print('Program:', i.getLiteral())
```

完成上面代码，启动解析器进行词法分析和语法分析，如下所示：

```
if __name__ == '__main__':  
    code = "let test = 123 ;"  
    Lex = Lexer(code)  
    Parser = CompilerParser(Lex)  
    Parser.startParser()
```

得到的输出结果如下：

```
Program: {'LetStatement': {'left': 'test', 'right': '123'}}
```

7.4 Babel 编译步骤

到目前为止，介绍了使用 Python 开发简单的解析器，将输入的源代码解析为 AST 抽象

语法树,了解了源代码到AST的转化过程。但是,从头开始实现编译器是极其复杂的,一个数字赋值语句就有上百行代码,所以可以使用一些现成的工具来帮助完成AST的解析,只需要在此基础上进行二次开发即可。

Babel是一个强大的JS编译器,确切地说是源码到源码的编译器,通常也叫作转换编译器(transpiler)。为Babel提供一些JS代码,Babel更改这些代码,然后返回新生成的代码。为Babel提供的代码会比较方便地转化为AST抽象语法树,之后只需要专注AST抽象语法树的转化生成。

Babel的编译过程主要有以下三个阶段。

- (1) 解析(Parse): 将输入字符流解析为AST抽象语法树。
- (2) 转化(Transform): 对抽象语法树进一步转化。
- (3) 生成(Generate): 根据转化后的语法树生成目标代码。

7.4.1 Babel的解析

Babel的解析实际上包含了两个内容:词法分析和语法分析。经过这一步后,可以直接得到输入源代码的AST抽象语法树,极大地方便了AST程序的开发。

7.4.2 Babel的转化

转化步骤接收AST并对其进行遍历,在此过程中对节点进行添加、更新及移除等操作。后续的混淆JS代码以及对混淆过的JS代码进行还原都在此处,这是本书的重点。之所以将字符流转化为抽象语法树,原因是树状结构更加容易进行原子操作,可以对任意的节点进行精细化处理。在抽象语法树中,代码间的关系被抽象为节点间的关系,而实现相同功能的节点之间的表示也是相同的。利用这一点,可以在语法树层面对输入的代码进行增、删、改、查,而不必关心具体的代码书写。制定几条规则,Babel就可以对抽象语法树进行遍历,完成整个代码的批量操作。

7.4.3 Babel的生成

代码生成步骤把最终(经过一系列转换之后)的AST转换成字符串形式的代码。代码生成其实很简单:深度优先遍历整个AST,然后构建可以表示转换后代码的字符串。经过这一步,就可以得到从AST抽象语法树层面修改过的代码。

7.5 小结

主要讲解AST抽象语法树的基本概念,介绍词法分析和语法分析,使用Python编写了简单的词法分析器和语法分析器,最后引入Babel编译器,帮助进行AST程序开发。

7.6 习题

1. 简述什么是 AST。
2. 简述编译器和解释器的主要区别。
3. 典型的编译器在逻辑功能上由哪几部分组成？
4. Babel 编译器在程序执行时，会包括以下哪些步骤？
 - A. 词法分析
 - B. 语法分析
 - C. 语义分析
 - D. 代码转化
5. 谈谈对 AST 程序开发的理解。

第8章

AST的API详解



JS 的语法非常灵活,直接混淆 JS 代码或者还原是很麻烦的,要考虑的情况太多,容易出错。但把 JS 代码转换成抽象语法树(以下简称 AST)后,一切都简单了。在编译原理里,从源代码到机器码,中间还需要经过很多步骤。例如,源代码通过词法分析器变为记号序列,接着通过语法分析器变为 AST,再通过语义分析器,一步步往下编译,最后变成机器码。所以,AST 实际上是一个概念性的东西,实现了词法分析器和语法分析器,就能把不同的语言解析成 AST。

如果想把 JS 代码转换成 AST,可以写代码,也可以使用现成的解析库。当使用的解析库不一样时,生成的 AST 会有所区别,本书采用的是 Babel 解析库。使用 AST 自动化处理 JS 代码前,需要对 AST 相关的 API 有一定的了解,这是本章着重介绍的内容。



视频讲解

8.1 AST 入门

8.1.1 AST 的基本结构

JS 代码解析成 AST 以后,类似于中间形式的 json 数据。经过 Babel 解析,里面的元素叫作节点(Node),同时 Babel 也提供了很多方法去操作这些节点。本节以一个案例来说明 AST 的基本结构,代码如下:

```
let obj = {
  name: 'xiaojianbang',
  add: function (a, b) {
    return a + b + 1000;
  },
  mul: function (a, b) {
    return a * b + 1000;
  },
};
```

本章后续内容中,把这段代码称为原始代码,如果没有特别说明,均以处理此代码为例。将上述代码在 AST Explorer 网页中进行解析,如图 8-1 所示。先选择 JS 语言以及@babel/parser 的解析方式,在网页的右上角会显示当前选择的解析方式。网页左边是输入 JS 代码的地方,右边是解析以后的 AST。当然该网页不单可以解析 JS 代码,还能够解析 HTML、CSS、Lua、PHP 等语言。

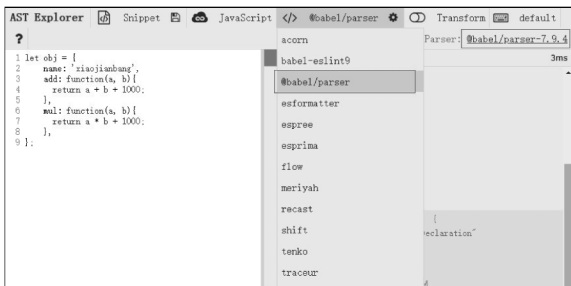


图 8-1 AST Explorer 网站

解析后 AST 有很多层级,这些层级互为父子节点。接下来逐级分析这些节点:

```
"body": [{  
  "type": "VariableDeclaration",  
  ...  
  "declarations": [{ ... }],  
  "kind": "let"  
}]
```

//注意,出于简化目的移除了某些属性

字符串形式的 type 字段表示节点的类型,例如上述代码中的 VariableDeclaration。每一种类型的节点都定义了一些附加属性,用来进一步描述该节点类型。VariableDeclaration 表示这是一个变量声明语句,kind 表示变量声明语句所使用的关键字,declarations 指声明的具体的变量。declarations 是一个数组,因为一个 let 关键字可以同时声明多个变量,如 let a,b; 当前只声明了一个变量 obj,所以 declarations 只有一个成员。再来看 declarations 中的内容:

```
{  
  "type": "VariableDeclarator",  
  ...  
  "id": {  
    "type": "Identifier",  
    ...  
    "name": "obj"  
  },  
  "init": { ... }  
}
```

declarations 里面是声明的具体变量的信息,每一个变量都以 VariableDeclarator 表示。VariableDeclarator 的属性主要是 id 和 init。id 很容易理解,表示这是一个 Identifier(标识符),name 是 obj,与原始代码一致。对于 init,假如只是声明变量,没有给初始值,那么 init 的值为 null,但在原始代码中是有初始值的,因此还要继续查看 init 中的内容:

```
{
  "type": "ObjectExpression",
  ...
  "properties": [{...},{...},{...}],
  "extra": {trailingComma: 150}
}
```

在原始代码中,把对象字面量赋值给了 obj,AST 中把这个称为 ObjectExpression(对象表达式)。有对象自然就有属性,还可以有多个属性,因此 properties 是一个数组,一个属性对应一个成员。接着查看 properties 中的第一个属性:

```
{
  "type": "ObjectProperty",
  ...
  "method": false,
  "key": {
    "type": "Identifier",
    ...
    "name": "name"
  },
  "computed": false,
  "shorthand": false,
  "value": {
    "type": "StringLiteral",
    ...
    "extra": {
      "rawValue": "xiaojianbang",
      "raw": "'xiaojianbang'"
    },
    "value": "xiaojianbang"
  }
}
```

type 为 ObjectProperty,表示该节点为对象属性。JS 中的对象是由一系列的键值对(key-value)组成的。由解析后的结果可知,key 是一个 Identifier(标识符),名字是 name。注意,对象的 key 也可以用字符串表示,这时 key 的节点类型就会变为 StringLiteral(字符串字面量)。value 节点的类型为 StringLiteral,具体的值是 xiaojianbang,但在该节点中,看到了三个 xiaojianbang。extra 节点是可以删掉的,如果是十六进制字符串、unicode 字符串或者 hex 形式的数值,这三个值就会不一样。接着继续查看第二个属性,如下所示:

```
{
  "type": "ObjectProperty",
  ...
  "method": false,
  "key": {
    "type": "Identifier",
    ...
    "name": "add"
  },
  "computed": false,
  "shorthand": false,
  "value": { ... }
}
```

这一层显示的这些属性,前面已经介绍过了,不再赘述。关键是 value 节点,因为在原始代码中把一个函数表达式赋值给了对象的属性。查看解析后的函数:

```
{
  "type": "FunctionExpression",
  ...
  "id": null,
  "generator": false,
  "async": false,
  "params": [{
    "type": "Identifier",
    ...
    "name": "a"
  }, {
    "type": "Identifier",
    ...
    "name": "b"
  }
],
  "body": {
    "type": "BlockStatement",
    ...
    "body": [{ ... }
  ],
  "directives": []
}
```

type 为 FunctionExpression(函数表达式)。函数名对应上述结构中的 id 节点,由于代码把一个匿名函数赋值给了 obj 的 add 属性,所以这里的 id 为 null。函数参数对应上述结构中的 params 节点,由于参数是可以有多个的,所以 params 是一个数组。注意,如果没有参数,params 值是一个空数组,而不是 null。函数体对应上述结构中的 body 节点。注意,这个 body 节点并非数组。一般函数体都会用 BlockStatement 节点包裹,BlockStatement 里的 body 节点是多条语句,所以这个 body 节点是一个数组。如果函数有'use strict'标记,

那么 directives 里面就会有相应的节点。

接着查看一下函数体中具体的语句。BlockStatement 里 body 节点的内容为：

```
[{
  "type": "ReturnStatement",
  ...
  "argument": {
    "type": "BinaryExpression",
    ...
    "left": {...},
    "operator": "+",
    "right": {
      "type": "NumericLiteral",
      ...
      "extra": {
        "rawValue": 1000,
        "raw": "1000"
      }
      "value": 1000
    }
  }
}]
```

在原始代码中,这个函数内只有一条 return 语句,对应这里的 ReturnStatement(返回语句)。argument 是 return 语句返回的内容。假如 return 语句没有返回内容,那么 argument 节点值为 null。在原始代码中,return 返回的是 $a+b+1000$,AST 解析成了 BinaryExpression(二项式)。JS 中的二元运算符都可以解析成 BinaryExpression。二项式主要分为三部分: left、operator 和 right。可以看出是把 $a+b$ 作为 left,1000 作为 right。对于 right 节点,type 为 NumericLiteral(数值字面量),value 为 1000。对于 left 节点,依然是一个二项式,代码如下:

```
{
  "type": "BinaryExpression",
  ...
  "left": {
    "type": "Identifier",
    ...
    "name": "a"
  },
  "operator": "+",
  "right": {
    "type": "Identifier",
    ...
    "name": "b"
  }
}
```

这里的二项式和之前的二项式相似。在原始代码中 obj 的第三个属性基本上与第二个

属性解析后的结果一致,这里不再赘述。

8.1.2 代码的基本结构

把原始代码保存成一个名为 demo.js 的文件,注意,需要保存成 utf-8 编码的格式。另外新建一个文件,用来解析 demo.js。AST 解析转换代码的基本结构如下所示:

```
const fs = require('fs');
const parser = require("@babel/parser");
const traverse = require("@babel/traverse").default;
const t = require("@babel/types");
const generator = require("@babel/generator").default;

const jscode = fs.readFileSync("./demo.js", {
  encoding: "utf-8"
});
let ast = parser.parse(jscode);

//在这里对 AST 进行一系列的操作

let code = generator(ast).code;
fs.writeFile('./demoNew.js', code, {err} => {});
```

fs 用来读写本地文件,通过 require 函数加载后赋值给 fs。@babel/parser 用来将 JS 代码转换成 AST,通过 require 函数加载后赋值给 parser。@babel/traverse 用来遍历 AST 中的节点,通过 require 函数加载后把其中的 default 赋值给 traverse。@babel/types 用来判断节点类型、生成新的节点等,通过 require 函数加载后赋值给 t。@babel/generator 用来把 AST 转换成 JS 代码,通过 require 函数加载后把其中的 default 赋值给 generator。

可以看出,AST 处理 JS 文件的基本步骤为:首先读取 JS 文件,解析成 AST,再对节点进行一系列的增、删、改、查操作,接着生成 JS 代码,最后保存到新文件中。后续内容中,如果没有特别说明,都以这个结构为准,代码只展示中间对 AST 有操作的部分。

8.2 Babel 中的组件

在 8.1.2 节的代码中,用到了 Babel 中的一些组件。这些组件各有不同的功能,本节介绍这部分内容。

8.2.1 parser 与 generator

这两个组件的作用是相反的。parser 组件用来将 JS 代码转换成 AST,generator 用来将 AST 转换成 JS 代码。

使用 let ast = parser.parse(jscode); 即可完成 JS 代码转换到 AST 的过程。这时输出 ast,就会跟网页中解析出一样的结构。输出前先使用 JSON.stringify 把对象转为 json,



视频讲解

数据,如 `console.log(JSON.stringify(ast, null, 2))`。另外,parser 的 `parse` 方法是有第二个参数的。

```
let ast = parser.parse(jscode, {
  sourceType: "module",
});
```

`sourceType` 默认为 `script`。当解析的 JS 代码中,含有 `'import'`、`'export'` 等关键字时,需要指定 `sourceType` 为 `module`,不然会有如下报错:

```
SyntaxError: 'import' and 'export' may appear only with 'sourceType: "module"' (1:0)
```

使用 `let code = generator(ast).code`; 可以把 AST 转换为 JS 代码。`generator` 返回的是一个对象,其中的 `code` 属性才是需要的代码。同时,`generator` 的第二个参数接收一个对象,可以设置一些选项来影响输出的结果。完整的选项介绍可在 Babel 官方文档 <https://babeljs.io/docs/en/babel-generator> 中查看,本节只介绍其中的一部分。

```
let code = generator(ast, {
  retainLines: false,
  comments: false,
  compact: true
}).code;
console.log(code);
```

`retainLines` 表示是否使用与源代码相同的行号,默认为 `false`,输出的是格式化后的代码。`comments` 表示是否保留注释,默认为 `true`。`compact` 表示是否压缩代码,与其相同作用的选项还有 `minified` 和 `concise`,只不过压缩的程度不一样,`minified` 压缩得最多,`concise` 压缩得最少。多个选项之间可以配合使用。

8.2.2 traverse 与 visitor

`traverse` 组件用来遍历 AST,简单说就是把 AST 上的各个节点都运行一遍,但单纯把节点都运行一遍是没有意义的,所以 `traverse` 需要配合 `visitor` 使用。

`visitor` 是一个对象,它可以定义一些方法,用来过滤节点。接下来用一个实际案例来讲解 `traverse` 和 `visitor` 的效果,代码如下:

```
let visitor = {};
visitor.FunctionExpression = function(path){
  console.log("xiaojianbang");
};
traverse(ast, visitor);
/* 输出
  xiaojianbang
  xiaojianbang
*/
```

首先声明对象,对象的名字可随意定义,再给对象增加一个名为 `FunctionExpression` 的方法,它的名字是需要遍历的节点类型,需要注意大小写。`traverse` 会遍历所有的节点,当节点类型为 `FunctionExpression` 时,调用 `visitor` 中相应的方法。如果想要处理其他节点类型,例如 `Identifier`,可以在 `visitor` 中继续定义方法,以 `Identifier` 命名即可。`visitor` 中的方法接收一个参数,`traverse` 在遍历时,会把当前节点的 `Path` 对象传给它,传过来的是 `Path` 对象而非节点(`Node`)。最后把 `visitor` 作为第二个参数传到 `traverse` 里,传给 `traverse` 的第一个参数是整个 `ast`。这段代码的意思是,从头开始遍历 `ast` 中的所有节点,过滤出 `FunctionExpression` 节点,执行相应的方法。在原始代码中,有两个 `FunctionExpression` 节点,因此,会输出两次 `xiaojianbang`。

定义 `visitor` 的方式有以下三种,最常用的是 `visitor2` 这种形式。

```
const visitor1 = {
  FunctionExpression: function(path){
    console.log("xiaojianbang");
  }
};
const visitor2 = {
  FunctionExpression(path){
    console.log("xiaojianbang");
  }
};
const visitor3 = {
  FunctionExpression:{
    enter(path){
      console.log("xiaojianbang");
    }
  }
};
```

在 `visitor3` 中,存在一个 `enter`。在遍历节点的过程中,有两次机会来访问一个节点,即进入节点时(`enter`)与退出节点时(`exit`)。以原始代码中的 `add` 函数为例,节点的遍历过程可描述如下:

```
进入 FunctionExpression
  进入 Identifier (params[0])走到尽头
  退出 Identifier (params[0])
  进入 Identifier (params[1])走到尽头
  退出 Identifier (params[1])
  进入 BlockStatement (body)
    进入 ReturnStatement (body)
      进入 BinaryExpression (argument)
        进入 BinaryExpression (left)
          进入 Identifier (left) 走到尽头
          退出 Identifier (left)
          进入 Identifier (right) 走到尽头
          退出 Identifier (right)
        退出 BinaryExpression (left)
```

```

        进入 NumericLiteral(right)走到尽头
        退出 NumericLiteral(right)
        退出 BinaryExpression (argument)
        退出 ReturnStatement (body)
        退出 BlockStatement (body)
        退出 FunctionExpression

```

正确选择节点处理时机,有助于提高代码效率。可以看出 traverse 是一个深度优先的遍历过程。因此,如果存在父子节点,那么 enter 的处理时机是先处理父节点,再处理子节点。而 exit 的处理时机是先处理子节点,再处理父节点。traverse 默认是在 enter 时处理,如果要在 exit 时处理,必须在 visitor 中写明。

```

const visitor3 = {
  FunctionExpression: {
    enter(path) {
      console.log("xiaojianbang enter");
    },
    exit(path) {
      console.log("xiaojianbang exit");
    }
  }
};
traverse(ast, visitor3);
/* 因为有两个函数,所以输出两次
   xiaojianbang enter
   xiaojianbang exit
   xiaojianbang enter
   xiaojianbang exit
*/

```

还可以把方法名用“|”连接成 FunctionExpression|BinaryExpression 形式的字符串,把同一个函数应用到多个节点,例如:

```

const visitor = {
  "FunctionExpression|BinaryExpression"(path){
    console.log("xiaojianbang");
  }
};
traverse(ast, visitor);

```

也可以把多个函数应用于同一个节点。原先是把一个函数赋值给 enter 或者 exit,现在改为函数的数组,会按顺序依次执行。示例代码如下:

```

function func1(path){
  console.log('func1');
}
function func2(path){

```

```
    console.log('func2');
  }
  const visitor = {
    FunctionExpression: {
      enter: [func1, func2]
    }
  };
  traverse(ast, visitor);
```

traverse 并非必须从头遍历,它可在任意节点向下遍历。例如,想要把代码中所有函数的第一个参数改为 x,代码如下:

```
const updateParamNameVisitor = {
  Identifier(path) {
    if (path.node.name === this.paramName) {
      path.node.name = "x";
    }
  }
};
const visitor = {
  FunctionExpression(path) {
    const paramName = path.node.params[0].name;
    path.traverse(updateParamNameVisitor, {
      paramName
    });
  }
};
traverse(ast, visitor);
```

这段代码先用 traverse 根据 visitor 去遍历所有节点。当得到 FunctionExpression 节点时,用 path.traverse 根据 updateParamNameVisitor 去遍历当前节点下的所有子节点,然后修改与函数第一个参数相同的标识符。在使用 path.traverse 时,还可以额外传入一个对象,在对应的 visitor 中用 this 去引用它。其中 path.node 才是当前节点,所以 path.node.params[0].name 可以取出函数的第一个参数名。

8.2.3 types 组件

该组件主要用来判断节点类型、生成新的节点等。判断节点类型的方法很简单,例如, t.isIdentifier(path, node),它等同于 path.node.type === "Identifier"。还可以在判断类型的同时附加条件,示例如下:

```
traverse(ast, {
  enter(path) {
    if (
      path.node.type === "Identifier" &&
```

```
        path.node.name === "n"
      ){
        path.node.name = "x";
      }
    }
  });
```

上述代码用来把标识符 `n` 改为 `x`，这是官方手册中的案例，但在实际修改中还需要考虑标识符的作用域。在这个案例中，`visitor` 没有做任何过滤，遍历到任何一个节点都调用 `enter` 函数，所以要判断类型为 `Identifier` 且 `name` 的值为 `n`，才修改为 `x`。这个案例可以等同地写为：

```
traverse(ast, {
  enter(path){
    if (
      t.isIdentifier(path.node, {name: 'n'})
    ){
      path.node.name = "x";
    }
  }
});
```

如果要判断其他类型，只需要更改 `is` 后面的类型。这些方法还可以归纳为：当节点不符合要求，会抛出异常而不是返回 `true` 或 `false`：

```
t.assertBinaryExpression(maybeBinaryExpressionNode);
t.assertBinaryExpression(maybeBinaryExpressionNode, { operator: "*" });
```

可以看出，`types` 组件中用于判断节点类型的函数是可以自己实现的，且过程也较为容易。因此，`types` 组件最主要的功能是可以方便地生成新的节点。接下来尝试用 `types` 组件来生成原始代码。注意，`Babel` 中的 API 有很多，不可能全部记住 API 的用法，一定要学会查看代码提示。

在原始代码中，最开始是一个变量声明语句，类型为 `VariableDeclaration`。因此，可以用 `t.variableDeclaration` 去生成它。在 `vscode` 中输入“`t.variableDeclaration`”，然后将鼠标指针悬停在 `variableDeclaration` 上，就会出现代码提示。也可以按 `Ctrl` 键，同时单击 `variableDeclaration`，跳转到一个以 `ts` 为后缀的文件中，有如下一段代码：

```
export function variableDeclaration(kind: "var" | "let" | "const", declarations: Array <
VariableDeclarator>): VariableDeclaration;
```

这段代码最后一个冒号后表示这个函数的返回值类型。括号里面的冒号前，是 `VariableDeclaration` 节点的属性。括号里面的冒号后，表示该参数允许传的类型。Array 表示这个参数是一个数组。因此，变量声明语句的生成代码可以写为：

```
let loa1Ast = t.variableDeclaration('let', [varDec]);
let code = generator(loa1Ast).code;
console.log(code);
```

要生成上述代码中的 varDec,需要先生成一个 VariableDeclarator 节点,表示变量声明的具体的值,在 ts 文件中的定义如下:

```
export function variableDeclarator(id: LVal, init?: Expression | null): VariableDeclarator;
```

VariableDeclarator 是该函数的返回值,id 和 init 是 VariableDeclarator 节点的属性。init 后面的问号,代表该参数可省略。根据 8.1.1 节的分析,这里的 id 是 Identifier 类型。生成 Identifier 的方法很简单,在 ts 文件中的定义如下:

```
export function identifier(name: any): Identifier;
```

在原始代码中对 obj 进行了初始化。所以这里的 init 是需要传值的。那么,生成 varDec 的代码可以写为:

```
let varDec = t.variableDeclarator(t.identifier('obj'), objExpr);
```

接着要生成 objExpr。这里需要一个 ObjectExpression,在 ts 文件中的定义如下:

```
export function objectExpression (properties: Array < ObjectMethod | ObjectProperty | SpreadElement >): ObjectExpression;
```

对象的属性可以有多个,所以需要数组。因此,生成 objExpr 的代码可以写为:

```
let objExpr = t.objectExpression([objProp1, objProp2, objProp3]);
```

在上述代码中,objProp1、objProp2 和 objProp3 都没有进行赋值。这里,还有一个新类型 ObjectProperty,在 ts 文件中的定义如下:

```
export function objectProperty(key: any, value: Expression | PatternLike, computed?: boolean, shorthand?: any, decorators?: Array<Decorator> | null): ObjectProperty;
```

key 的值在原始代码中为 name,它是一个 Identifier。后面三个参数都是可选的,这里都选择不传入。其中,节点属性 computed 将在后续内容中介绍。value 表示对象属性的具体的值。在原始代码中,第 1 个属性的值是一个字符串字面量,用 StringLiteral 表示。第 2 个和第 3 个属性的值都为函数表达式,用 FunctionExpression 表示。StringLiteral 在 ts 文件中的定义如下:

```
export function stringLiteral(value: string): StringLiteral;
```

因此,生成 obj 三个属性的代码为:

```
let objProp1 = t.objectProperty(t.identifier('name'), t.stringLiteral('xiaojianbang'));
let objProp2 = t.objectProperty(t.identifier('add'), funcExpr2);
let objProp3 = t.objectProperty(t.identifier('mul'), funcExpr3);
```

上述代码中,funcExpr2 和 funcExpr3 还没有进行赋值。接着介绍 FunctionExpression 在 ts 文件中的定义,id 表示函数名,params 表示参数列表,body 用 BlockStatement 包裹所有语句,其余参数可选,代码如下:

```
export function functionExpression(id: Identifier | null | undefined, params: Array <
Identifier | Pattern | RestElement | TSPParameterProperty>, body: BlockStatement, generator?:
boolean, async?: boolean): FunctionExpression;

export function blockStatement(body: Array < Statement >, directives?: Array < Directive >):
BlockStatement;
```

在原始代码中都是由匿名函数直接赋值给 obj 的属性。因此,这里 id 为 null,params 列表用 t.identifier 生成,BlockStatement 节点用 t.blockStatement 生成,代码如下:

```
let a = t.identifier('a');
let b = t.identifier('b');
let bloSta2 = t.blockStatement([retSta2]);
let bloSta3 = t.blockStatement([retSta3]);
let funcExpr2 = t.functionExpression(null, [a, b], bloSta2);
let funcExpr3 = t.functionExpression(null, [a, b], bloSta3);
```

上述代码中,retSta2 和 retSta3 还需要另外生成。特别说明的是,如果要生成一个空函数,即函数体为空,则 blockStatement 的参数给空数组,而不是 null。原始代码中,两个函数内都含有返回语句、二项式和数值字面量。在 AST 中可以分别使用 ReturnStatement、BinaryExpression 和 NumericLiteral 来表示。它们在 ts 文件中的定义为:

```
export function returnStatement(argument?: Expression | null): ReturnStatement;

export function binaryExpression(operator: "+" | "-" | "/" | "%" | "*" | "**" | "&" | "|" |
">>" | ">>>" | "<<" | "^" | "==" | "===" | "!=" | "!==" | "in" | "instanceof" | ">" | "<" |
">=" | "<=", left: Expression, right: Expression): BinaryExpression;

export function numericLiteral(value: number): NumericLiteral;
```

接下来,把代码中剩余的部分生成完毕:

```
let a = t.identifier('a');
let b = t.identifier('b');
let binExpr2 = t.binaryExpression("+", a, b);
let binExpr3 = t.binaryExpression("*", a, b);
let retSta2 = t.returnStatement(t.binaryExpression("+", binExpr2, t.numericLiteral(1000)));
```



```
let retSta3 = t.returnStatement(t.binaryExpression(" + ", binExpr3, t.numericLiteral(1000)));  
let bloSta2 = t.blockStatement([retSta2]);  
let bloSta3 = t.blockStatement([retSta3]);
```

完整的代码,以及执行之后的结果如下所示:

```
let a = t.identifier('a');  
let b = t.identifier('b');  
let binExpr2 = t.binaryExpression(" + ", a, b);  
let binExpr3 = t.binaryExpression(" * ", a, b);  
let retSta2 = t.returnStatement(t.binaryExpression(" + ", binExpr2, t.numericLiteral(1000)));  
let retSta3 = t.returnStatement(t.binaryExpression(" + ", binExpr3, t.numericLiteral(1000)));  
let bloSta2 = t.blockStatement([retSta2]);  
let bloSta3 = t.blockStatement([retSta3]);  
let funcExpr2 = t.functionExpression(null, [a, b], bloSta2);  
let funcExpr3 = t.functionExpression(null, [a, b], bloSta3);  
let objProp1 = t.objectProperty(t.identifier('name'), t.stringLiteral('xiaojianbang'));  
let objProp2 = t.objectProperty(t.identifier('add'), funcExpr2);  
let objProp3 = t.objectProperty(t.identifier('mul'), funcExpr3);  
let objExpr = t.objectExpression([objProp1, objProp2, objProp3]);  
let varDec = t.variableDeclarator(t.identifier('obj'), objExpr);  
let loca1Ast = t.variableDeclaration('let', [varDec]);  
let code = generator(loca1Ast).code;  
console.log(code);  
/* 输出结果  
let obj = {  
  name: "xiaojianbang",  
  add: function (a, b) {  
    return a + b + 1000;  
  },  
  mul: function (a, b) {  
    return a * b + 1000;  
  }  
};  
*/
```

在JS代码处理转换过程中,生成的新节点一般会添加或替换到已有的节点中。在8.3.2节中会有详细介绍。

上述案例中,用到了StringLiteral和NumericLiteral,同时在Babel中还定义了一些其他的字面量。

```
export function nullLiteral(): NullLiteral;  
export function booleanLiteral(value: boolean): BooleanLiteral;  
export function regExpLiteral(pattern: string, flags?: any): RegExpLiteral;
```

因此,不同的字面量需要调用不同的方法生成。当生成比较多的字面量时,难度会不断上升。其实在 Babel 中还提供了 `valueToNode`,如下所示:

```
export function valueToNode(value: undefined): Identifier
export function valueToNode(value: boolean): BooleanLiteral
export function valueToNode(value: null): NullLiteral
export function valueToNode(value: string): StringLiteral
export function valueToNode ( value: number ): NumericLiteral | BinaryExpression
| UnaryExpression
export function valueToNode(value: RegExp): RegExpLiteral
export function valueToNode(value: ReadonlyArray<undefined | boolean | null | string | number
| RegExp | object>): ArrayExpression
export function valueToNode(value: object): ObjectExpression
export function valueToNode(value: undefined | boolean | null | string | number | RegExp |
object): Expression
```

由此可以看出,`valueToNode` 可以很方便地生成各种类型。除了原始类型 `undefined`、`null`、`string`、`number` 和 `boolean`,还可以是对象类型 `RegExp`、`ReadonlyArray` 和 `object`,示例代码如下:

```
let loaclAst = t.valueToNode([1, "2", false, null, undefined, /\w\s/g, {x: '1000', y:
2000}]);
let code = generator(loaclAst).code;
console.log(code);
/* 输出结果
    [1, "2", false, null, undefined, /\w\s/g, {
      x: "1000",
      y: 2000
    }]
  */
```

结合 `ts` 文件中的定义和 AST 解析后的 `json` 数据,可以迅速掌握这些 API 的使用方法。

前面的介绍中提到了 Babel 解析后的 AST,其实它是一段 `json` 数据。因此,也可以按照 AST 的结构来构造一段 `json` 数据,以此生成想要的代码,但比使用 `types` 组件麻烦。

```
let obj = {};
obj.type = 'BinaryExpression';
obj.left = {type: 'NumericLiteral', value: 1000};
obj.operator = '/';
obj.right = {type: 'NumericLiteral', value: 2000};
let code = generator(obj).code;
console.log(code);
//输出 1000 / 2000
```

8.3 Path 对象详解

8.3.1 Path 与 Node 的区别

下面以一个案例来清楚地说明问题。其中 `path.stop` 用来停止遍历节点, Babel 中的 `path.skip` 效果与之类似。这段代码的意思是, 当在函数表达式中遍历到一个 `Identifier` 节点时, 就输出 `path`, 然后停止遍历。根据 `traverse` 遍历规则, 遍历到的这个 `Identifier` 节点肯定是 `params[0]`。示例代码如下:



视频讲解

```
const updateParamNameVisitor = {
  Identifier(path) {
    if (path.node.name === this.paramName) {
      path.node.name = "x";
    }
    console.log(path);
    path.stop();
  }
};

const visitor = {
  FunctionExpression(path) {
    const paramName = path.node.params[0].name;
    path.traverse(updateParamNameVisitor, {
      paramName
    });
  }
};

traverse(ast, visitor);
/*
  NodePath {
    parent: Node {...}
    hub: undefined,
    contexts: [...]
    data: null,
    _traverseFlags: 0,
    state: {...}
    opts: {...}
    skipKeys: null,
    parentPath: NodePath {...}
    context: TraversalContext {...}
    container: [...]
    listKey: 'params',
    key: 0,
    node: Node {
      type: 'Identifier',
      ...
      name: 'a'
    }
  }
```

```
    },  
    scope: Scope {...}  
    type: 'Identifier'  
  }  
* /
```

`path.node` 能取出 `Identifier` 所在的 `Node` 对象,该对象与 `AST Explorer` 网页中解析出来的 `AST` 节点结构一致。简单来说,节点是生成 `JS` 代码的“原料”,是 `Path` 中的一部分。`Path` 是一个对象,用来描述两个节点之间的连接。`Path` 除了具有上述显示的这些属性以外,还包含添加、更新、移动和删除节点等很多有关的方法。

8.3.2 Path 中的方法

理清了 `Path` 与 `Node` 的关系后,再来学习 `Path` 中的方法。

1. 获取子节点/Path

为了得到 `AST` 节点的属性值,一般会先访问到该节点,然后利用 `path.node`, `property` 方法获取属性,如下所示:

```
const visitor = {  
  BinaryExpression(path) {  
    console.log(path.node.left);  
    console.log(path.node.right);  
    console.log(path.node.operator);  
  }  
};  
traverse(ast, visitor);  
/*  
  Node {type: 'BinaryExpression', ..., left: Node}  
  Node {type: 'NumericLiteral', ..., extra: { ... }}  
  +  
  Node {type: 'Identifier', ..., name: 'a'}  
  Node {type: 'Identifier', ..., name: 'b'}  
  +  
* /
```

通过这个方法获取到的是 `Node` 或者具体的属性值,`Node` 不能使用 `Path` 相关的方法。如果想要获取到该属性的 `Path`,就需要使用 `Path` 对象的 `get`,传递的参数为 `key`(即该属性名的字符串形式)。如果是多级访问,那么以点连接多个 `key`,例如:

```
const visitor = {  
  BinaryExpression(path) {  
    console.log(path.get('left.name'));  
    console.log(path.get('right'));  
    console.log(path.get('operator'));  
  }  
};
```

```
    }  
  };  
  traverse(ast, visitor);  
  /* 会输出 n 个 NodePath  
      NodePath {parent: Node, hub: undefined, contexts: Array(0), data: null, _traverseFlags: 0}  
  */  
  */
```

可以看到,任何形式的属性值,通过 Path 对象的 get 去获取,都会包装成 Path 对象再返回。不过,像 operator、name 这一类的属性值,没有必要包装成 Path 对象。

2. 判断 Path 类型

回顾 8.3.1 节中输出的 Path 对象,会发现最后有一个 type 属性,它与 Node 中的 type 基本一致。Path 对象提供相应的方法来判断自身类型,使用方法与 types 组件差不多,只不过 types 组件判断的是 Node。解析下原始代码中的第一个函数中的二项式, left 是 $a+b$, right 是 1000。因此,下面的代码执行以后依次输出 false、true 和报错,如下所示:

```
const visitor = {  
  BinaryExpression(path) {  
    console.log(path.get('left').isIdentifier());  
    console.log(path.get('right').isNumericLiteral({  
      value: 1000  
    }));  
    path.get('left').assertIdentifier();  
  }  
};  
traverse(ast, visitor);  
/* 输出  
    false  
    true  
    报错  
  */
```

3. 节点转代码

当代码比较复杂时,就要用动态调试。在 vscode 中调试,只需要单击行号,设置断点即可。启动调试后,代码运行到断点处会停止运行。这里的调试与调试普通 JS 文件无异,还可以适时地插入 console 来排查错误,也可以基于当前节点生成代码来排查错误。即很多时候需要在执行过程中把部分节点转为代码,而不是在最后才把整个 AST 转成代码。generator 组件也可以把 AST 中的一部分节点转成代码,这对节点遍历过程中的调试很有帮助。示例代码如下:

```
const visitor = {  
  FunctionExpression(path) {  
    console.log(generator(path.node).code);  
    //console.log( path.toString() );  
  }  
};
```

```
        //console.log( path + ' ');
    }
};
traverse(ast, visitor);
/*
    function (a, b) {
        return a + b + 1000;
    }
    function (a, b) {
        return a * b + 1000;
    }
*/
*/
```

Path 对象复写了 Object 的 toString。在 Path 对象的 toString 中,调用了 generator 组件把节点转为代码。因此,可以用 path.toString 把节点转为字符串,也可以用 path + '' 来隐式转成字符串。

4. 替换节点属性

替换节点属性与获取节点属性方法相同,只是改为赋值。但也并非随意替换,需要注意的是,替换的类型要在允许的类型范围内。因此需要熟悉 AST 的结构,如下所示:

```
const visitor = {
  BinaryExpression(path) {
    path.node.left = t.identifier("x");
    path.node.right = t.identifier("y");
  }
};
traverse(ast, visitor);
/*
    let obj = {
      name: 'xiaojianbang',
      add: function (a, b) {
        return x + y;
      },
      mul: function (a, b) {
        return x * y;
      }
    };
*/
```

5. 替换整个节点

Path 对象中与替换相关的方法有 replaceWith、replaceWithMultiple、replaceInline 和 replaceWithSourceString。

replaceWith 是用一个节点替换另一个节点,并且是严格的一换一。示例代码如下:

```
const visitor = {
  BinaryExpression(path) {
```

```
        path.replaceWith(t.valueToNode('xiaojianbang'));
    }
};
traverse(ast, visitor);
/*
    let obj = {
      name: 'xiaojianbang',
      add: function (a, b) {
        return "xiaojianbang";
      },
      mul: function (a, b) {
        return "xiaojianbang";
      }
    };
*/
```

replaceWithMultiple 也是用节点换节点,不过是多换一。示例代码如下:

```
const visitor = {
  ReturnStatement(path) {
    path.replaceWithMultiple([
      t.expressionStatement(t.stringLiteral("xiaojianbang")),
      t.expressionStatement(t.numericLiteral(1000)),
      t.returnStatement(),
    ]);
    path.stop();
  }
};
traverse(ast, visitor);
/*
    let obj = {
      name: 'xiaojianbang',
      add: function (a, b) {
        "xiaojianbang";
        1000;
        return;
      },
      mul: function (a, b) {
        "xiaojianbang";
        1000;
        return;
      }
    };
*/
```

上述代码中有两处要特别说明:当表达式语句单独在一行时(没有赋值),最好用 expressionStatement 包裹;替换后的节点, traverse 也是能遍历到的,因此替换时要极其小心,否则容易造成不合理的递归调用。例如上述代码,把 return 语句进行替换,但是替换的语句里又有 return 语句,就会陷入死循环。解决方法是加入 path. stop, 替换完成之后,立

刻停止遍历当前节点和后续的子节点。

`replaceInline` 接收一个参数。如果参数不为数组,那么 `replaceInline` 等同于 `replaceWith`; 如果参数是一个数组,那么 `replaceInline` 等同于 `replaceWithMultiple`,其中的数组成员必须都是节点。示例代码如下:

```
const visitor = {
  StringLiteral(path) {
    path.replaceInline(
      t.stringLiteral('Hello AST!'));
    path.stop();
  },
  ReturnStatement(path) {
    path.replaceInline([
      t.expressionStatement(t.stringLiteral("xiaojianbang")),
      t.expressionStatement(t.numericLiteral(1000)),
      t.returnStatement(),
    ]);
    path.stop();
  }
};
traverse(ast, visitor);
/*
  let obj = {
    name: "Hello AST!",
    add: function (a, b) {
      "xiaojianbang";
      1000;
      return;
    },
    mul: function (a, b) {
      "xiaojianbang";
      1000;
      return;
    }
  };
*/
```

上述代码中,visitor 中的函数也要加入 `path.stop`,原因和之前介绍的等同。

最后看 `replaceWithSourceString` 的用法,该方法用字符串源代码替换节点,如把原始代码中的函数改为闭包形式,示例代码如下:

```
traverse(ast, {
  ReturnStatement(path) {
    let argumentPath = path.get('argument');
    argumentPath.replaceWithSourceString(
      'function(){return ' + argumentPath + '}()');
  };
  path.stop();
});
```



```
    }  
  });  
  /*  
    let obj = {  
      name: 'xiaojianbang',  
      add: function (a, b) {  
        return function () {  
          return a + b + 1000;  
        }();  
      },  
      mul: function (a, b) {  
        return function () {  
          return a * b + 1000;  
        }();  
      }  
    };  
  */  
*/
```

首先遍历 `ReturnStatement`, 然后通过 `Path` 的 `get` 获取子 `Path`, 才能调用 `Path` 的相关方法去操作 `argument` 节点。同时, `replaceWithSourceString` 替换后的节点也会被解析, 也就是说会被 `traverse` 遍历到。因为里面也有 `return` 语句, 所以需要加上 `path. stop`。上述代码中还用节点转代码, 只不过是隐式转换。

凡是需要修改节点的操作, 都推荐使用 `Path` 对象的方法。当调用一个修改节点的方法后, `Babel` 会更新 `Path` 对象。

6. 删除节点

示例代码如下:

```
traverse(ast, {  
  EmptyStatement(path){  
    path.remove();  
  }  
});
```

`EmptyStatement` 指的是空语句, 就是多余的分号。使用 `path.remove` 删除当前节点。

7. 插入节点

想要把节点插入到兄弟节点中, 可以使用 `insertBefore` 和 `insertAfter` 分别在当前节点的前后插入节点。代码如下:

```
traverse(ast, {  
  ReturnStatement(path) {  
    path.insertBefore(t.expressionStatement(t.stringLiteral("Before")));  
    path.insertAfter(t.expressionStatement(t.stringLiteral("After")));  
  }  
});  
/*
```

```

let obj = {
  name: 'xiaojianbang',
  add: function (a, b) {
    "Before";
    return a + b + 1000;
    "After";
  },
  mul: function (a, b) {
    "Before";
    return a * b + 1000;
    "After";
  }
};
*/

```

在上述代码中,如果只想操作某一个函数中的 ReturnStatement,可以在 visitor 的函数中进行判断,不符合要求的直接 return 即可。需要注意,使用 path.stop 是不可行的。



视频讲解

8.3.3 父级 Path

回顾 8.3.2 节中输出的 Path 对象,可以看到有 parentPath 和 parent 两个属性。其中 parentPath 类型为 NodePath,所以它是父级 Path。parent 类型为 Node,所以它是父节点。只要获取到父级 Path,就可以调用 Path 对象的各种方法去操作父节点。父级 Path 的获取可以使用 path.parentPath。path.parentPath.node 等同于 path.parent,也就是说 parent 是 parentPath 中的一部分。

1. path.findParent

个别情况下,需要从一个路径向上遍历语法树,直到满足相应的条件。这时可以使用 Path 对象的 findParent,示例如下:

```

traverse(ast, {
  ReturnStatement(path) {
    console.log(path.findParent((p) => p.isObjectExpression()));
    //path.findParent(function(p){return p.isObjectExpression();});
  }
});

```

Path 对象的 findParent 接收一个回调函数,在向上遍历每一个父级 Path 时,会调用该回调函数,并传入对应的父级 Path 对象作为参数。当该回调函数返回真值时,则将对应的父级 Path 返回。上述代码会遍历 ReturnStatement,然后向上找父级 Path,当找到 Path 对象类型为 ObjectExpression 的情况时,就返回该 Path 对象。

2. path.find

这个方法使用场景较少,使用方法与 findParent 一致,只不过 find 方法查找的范围包含当前节点,而 findParent 不包含。

```
traverse(ast, {
  ObjectExpression(path){
    console.log( path.find((p) => p.isObjectExpression()) );
  }
});
```

3. path.getFunctionParent

向上查找与当前节点最接近的父函数 path.getFunctionParent,返回的也是 Path 对象。

4. path.getStatementParent

向上遍历语法树直到找到语句父节点。例如,声明语句、return 语句、if 语句、witch 语句和 while 语句等,返回的也是 Path 对象。该方法从当前节点开始找,如果想要找到 return 语句的父语句,就需要从 parentPath 中去调用,代码如下:

```
traverse(ast, {
  ReturnStatement(path){
    console.log( path.parentPath.getStatementParent() );
  }
});
```

5. 父级 Path 的其他方法

其他方法的使用与之前介绍的类似,如替换父节点 path.parentPath.replaceWith (Node)和删除父节点 path.parentPath.remove 等。

8.3.4 同级 Path

在介绍同级 Path 之前,需要先介绍下容器(container)。先来看以下这个例子:

```
traverse(ast, {
  ReturnStatement(path){
    console.log(path);
  }
});
/*
NodePath {
  parent: Node {...},
  ...
  parentPath: NodePath {...},
  ...
  container: [
    Node {
      type: 'ReturnStatement',
      ...
      argument: [Node]
    }
  ]
}
```



视频讲解

```
    ],  
    listKey: 'body',  
    key: 0,  
    node: Node {...},  
    scope: Scope {...},  
    type: 'ReturnStatement'  
  }  
  /*
```

上述代码遍历 ReturnStatement 节点,并直接输出 Path 对象。之前介绍的 AST 结构, ReturnStatement 是放在 BlockStatement 的 body 节点中的,并且该 body 节点是一个数组。输出的 Path 对象中的几个关键属性里,container 就是容器,在这个例子中它是一个数组,里面只有一个 ReturnStatement 节点,与原始代码吻合。listKey 是容器名, ReturnStatement 是放在 BlockStatement 的 body 节点中的,因此把 body 节点当作容器。

接下来介绍 key。在上述代码输出的 Path 对象中,可以看到有一个 key 属性,这个 key 就是之前介绍的 path.get 方法的参数。实际上它就是容器对象的属性名,或者说是容器数组的索引。这里的容器是一个数组,key 代表当前节点在容器中的位置。

并非只有 body 节点才是容器。再来看以下这个例子:

```
traverse(ast, {  
  ObjectProperty(path){  
    console.log(path);  
  }  
});  
/*  
  NodePath {  
    parent: Node {...},  
    ...  
    parentPath: NodePath {...},  
    ...  
    container: [  
      Node {  
        type: 'ObjectProperty',  
        ...  
        method: false,  
        key: [Node],  
        computed: false,  
        shorthand: false,  
        value: [Node]  
      },  
      Node {...},  
      Node {...}  
    ],  
    listKey: 'properties',  
    key: 0,  
    node: Node {...},  
    scope: Scope {...},
```

```
    type: 'ObjectProperty'
  }
  * /
```

上述代码遍历 `ObjectProperty` 节点,然后直接输出 `Path` 对象。`ObjectProperty` 是在 `ObjectExpression` 的 `properties` 属性中的。查看 `Path` 对象中的几个关键属性, `container` 是容器, `listKey` 是容器名。在原始代码中,有三个 `ObjectProperty`, 对应容器中的三个 `Node` 对象。 `key` 为 0, 表示当前节点是容器中索引为 0 的成员, 也就是说容器中的节点互为兄弟(同级)节点。

`container` 并非一直都是数组,例如以下这个例子:

```
traverse(ast, {
  ObjectExpression(path){
    console.log(path);
  }
});
/*
  NodePath {
    parent: Node {...},
    ...
    parentPath: NodePath {...},
    ...
    container: Node{
      type: 'VariableDeclarator',
      ...
      id: Node {
        type: 'Identifier',
        ...
        name: 'obj'
      },
      init: Node {
        type: 'ObjectExpression',
        ...
        properties: [Array],
        extra: [Object]
      }
    },
    listKey: undefined,
    key: 'init',
    node: Node {...},
    scope: Scope {...},
    type: 'ObjectExpression'
  }
  * /
```

在上述代码中, `container` 是一个 `Node` 对象, `listKey` 为 `undefined`, 其实可以说它没有容器, 也就是没有兄弟(同级)节点。在原始代码解析后的 `AST` 结构中, `ObjectExpression`

是 VariableDeclarator 的初始化值 (init 节点)。此时 key 不是数组下标, 而是对象的属性名。

了解容器之后, 接着介绍同级 Path 相关的属性和方法。一般 container 为数组时就有同级节点, 以下内容只考虑 container 为数组的情况, 只有这种情况才有意义。示例代码如下:

```
traverse(ast, {
  ReturnStatement(path) {
    console.log( path.inList );      //true
    console.log( path.container );   // [node {type: 'ReturnStatement' ... }]
    console.log( path.listKey );     // body
    console.log( path.key );         // 0
    console.log( path.getSibling(path.key) );
    // node {type: 'ReturnStatement' ... }
  }
});
```

1. path.inList

用于判断是否有同级节点。注意, 当 container 为数组, 但只有一个成员时, 会返回 true。

2. path.key, path.container, path.listKey

使用 path.key 获取当前节点在容器中的索引。使用 path.container 获取容器 (包含所有同级节点的数组)。使用 path.listKey 获取容器名。

3. path.getSibling(index)

它用于获取同级 Path, 其中参数 index 为容器数组中的索引。index 可以通过 path.key 来获取。可以对 path.key 进行加减操作来定位到不同的同级 Path。

4. unshiftContainer 与 pushContainer

示例代码如下:

```
traverse(ast, {
  ReturnStatement(path) {
    path.parentPath.unshiftContainer('body', [
      t.expressionStatement(t.stringLiteral('Before1')),
      t.expressionStatement(t.stringLiteral('Before2'))]);
    console.log(path.parentPath.pushContainer('body',
      t.expressionStatement(t.stringLiteral('After'))));
  }
});
/*
let obj = {
  name: 'xiaojianbang',
  add: function (a, b) {
    "Before1";
    "Before2";
  }
};
```

```
        return a + b + 1000;
        "After";
    },
    mul: function (a, b) {
        "Before1";
        "Before2";
        return a * b + 1000;
        "After";
    }
};
* /
```

从上述代码中,可以看出 `unshiftContainer` 往容器最前面加入节点,`pushContainer` 往容器最后面加入节点。它们在 `ts` 文件中的定义如下:

```
pushContainer <Nodes extends Node | Node[]>(listKey: ArrayKeys <T>, nodes: Nodes): NodePaths <Nodes>;
unshiftContainer <Nodes extends Node | Node[]>(listKey: ArrayKeys <T>, nodes: Nodes): NodePaths <Nodes>;
```

可以看出,第一个参数给 `listKey`,第二个参数给 `Nodes`。`Nodes` 是 `Nodes extends Node | Node[]`,因此可以给 `Node` 或者 `Node` 的数组,最后函数返回加入的 `Nodes` 的 `Path` 对象。

8.4 scope 详解

`scope` 提供了一些属性和方法,可以方便地查找标识符的作用域,获取并修改标识符的所有引用,以及判断标识符是否为参数或常量。如果不是常量,也可以知道在哪里修改了它。本节以下面的代码为例:

```
const a = 1000;
let b = 2000;
let obj = {
    name: 'xiaojianbang',
    add: function (a) {
        a = 400;
        b = 300;
        let e = 700;
        function demo() {
            let d = 600;
        }
        demo();
        return a + a + b + 1000 + obj.name;
    }
};
obj.add(100);
```



视频讲解

在上述代码中,有一个 add,并且在 add 中又定义了一个 demo。以这种方式定义的 demo,在 AST 中的类型为 FunctionDeclaration。

8.4.1 获取标识符作用域

scope、block 属性可以用来获取标识符作用域,返回 Node 对象。使用方法分为两种情况:变量和函数。以下为标识符为变量的情况,如下所示:

```
traverse(ast, {
  Identifier(path) {
    if (path.node.name === 'e') {
      console.log(generator(path.scope.block).code);
    }
  }
});
/*
function (a) {
  a = 400;
  b = 300;
  let e = 700;
  function demo() {
    let d = 600;
  }
  demo();
  return a + a + b + 1000 + obj.name;
}
*/
```

既然 path.scope.block 返回 Node 对象,那么就可以使用 generator 来生成代码。上述代码遍历所有 Identifier,当名字为 e 时,把当前节点的作用域转代码。变量 e 是定义在 add 内部的,作用域范围是整个 add。但是如果遍历的是一个函数,它的作用域会有些特别。来看下面这个例子:

```
traverse(ast, {
  FunctionDeclaration(path) {
    console.log(generator(path.scope.block).code);
  }
});
/*
function demo() {
  let d = 600;
}
*/
```

上述代码遍历 FunctionDeclaration,在原始代码中只有 demo 符合要求,但是 demo 的作用域实际上应该是整个 add 的范围。因此输出与实际不符,这时需要去获取父级作用域。获取函数的作用域代码如下:


```
traverse(ast, {
  FunctionDeclaration(path) {
    console.log(generator(path.scope.parent.block).code);
  }
});
/*
function (a) {
  a = 400;
  b = 300;
  let e = 700;
  function demo() {
    let d = 600;
  }
  demo();
  return a + a + b + 1000 + obj.name;
}
*/
```

8.4.2 scope.getBinding

scope.getBinding 接收一个类型为 string 的参数,用来获取对应标识符的绑定。为了更直观地说明绑定的含义,先来看下面这段代码。遍历 FunctionDeclaration,符合要求的只有 demo,然后获取当前节点下的绑定 a,直接输出 binding,代码如下:

```
traverse(ast, {
  FunctionDeclaration(path) {
    let binding = path.scope.getBinding('a');
    console.log(binding);
  }
});
/*
Binding {
  identifier: Node {type: 'Identifier', ... name: 'a'},
  scope: Scope{ ...
    block: Node {type: 'FunctionExpression', ... } ... },
  path: NodePath {...},
  kind: 'param',
  constantViolations: [...],
  constant: false,
  referencePaths: [...],
  referenced: true,
  references: 2,...}
*/
```

getBinding 中传的值必须是当前节点能够引用到的标识符名。如传入 g,这个标识符并不存在,或者说当前节点引用不到,那么 getBinding 会返回 undefined。

接下来介绍 Binding 中关键的属性。identifier 是 a 标识符的 Node 对象。path 是 a 标识符的 Path 对象。kind 中表明了这是一个参数,但它并不代表就是当前 demo 的参数。实际上在原始代码中,a 是 add 的参数(当函数中局部变量与全局变量重名时,使用的是局部变量)。constant 表示是否常量。referenced 表示当前标识符是否被引用。references 表示当前标识符被引用的次数。constantViolations 与 referencePaths 会在后续内容中单独讲述。

另外可以看出,Binding 中也有 scope。因为获取的是 a 的 Binding,所以是 a 的 scope。将其中的 block 节点转为代码后,可以看出它的作用域范围就是 add。值得一提的是,假如获取的是 demo 的 Binding,将其中的 block 节点转为代码后,输出的也是 add。因此,获取函数作用域也可以用如下方式:

```
traverse(ast, {
  FunctionExpression(path) {
    let bindingA = path.scope.getBinding('a');
    let bindingDemo = path.scope.getBinding('demo');
    console.log(bindingA.referenced);
    console.log(bindingA.references);
    console.log(generator(bindingA.scope.block).code);
    console.log(generator(bindingDemo.scope.block).code);
  }
});
/*
true
2
输出两次
function (a) {
  a = 400;
  b = 300;
  let e = 700;
  function demo() {
    let d = 600;
  }
  demo();
  return a + a + b + 1000 + obj.name;
}
*/
```

8.4.3 scope.getOwnBinding

该函数用于获取当前节点自己的绑定,也就是不包含父级作用域中定义的标识符的绑定。但是该函数会得到子函数中定义的标识符的绑定,来看下面这个例子:

```
function TestOwnBinding(path){
  path.traverse({
    Identifier(p){
```

```

        let name = p.node.name;
        console.log( name, !!p.scope.getOwnBinding(name) );
    }
    });
}
traverse(ast, {
    FunctionExpression(path){
        TestOwnBinding(path);
    }
});
/*
a true
a true
b false
e true
demo false
d true
demo true
a true
a true
b false
obj false
name false
*/

```

上述代码遍历 FunctionExpression 节点,当前案例中符合要求的只有 add 函数,然后遍历该函数下所有 Identifier,输出标识符名和 getOwnBinding 的结果。查看输出结果,可以发现子函数 demo 中定义的 d 变量,该变量也可以通过 getOwnBinding 得到。也就是说,如果只想获取当前节点下定义的标识符,而不涉及子函数的话,还需要进一步判断。可以通过判断标识符作用域是否与当前函数一致来确定,如:

```

function TestOwnBinding(path){
    path.traverse({
        Identifier(p){
            let name = p.node.name;
            let binding = p.scope.getBinding(name);
            binding && console.log( name, generator(binding, scope.block).code == path + '' );
        }
    });
}
traverse(ast, {
    FunctionExpression(path){
        TestOwnBinding(path);
    }
});
/*
a true

```

```
a true
b false
e true
demo true
d false
demo true
a true
a true
b false
obj false
* /
```

上述代码通过 binding、scope、block 获取标识符作用域,转为代码后,再与当前节点的代码比较,就可以确定是否是当前函数中定义的标识符。因为子函数中定义的标识符,作用域范围是子函数本身,添加判断作用域的代码后,使用 getBinding 和 getOwnBinding 得到的结果是一样的。

8.4.4 referencePaths 与 constantViolations

首先介绍 referencePaths。假如标识符被引用,referencePaths 中会存放所有引用该标识符的节点的 Path 对象。查看 referencePaths 中的内容,如下所示:

```
referencePaths: [
  NodePath {
    parent: Node { type: 'BinaryExpression', ... }, ...
    parentPath: NodePath { ... type: 'BinaryExpression', ...
    container: Node { type: 'BinaryExpression', ... },
    listKey: undefined,
    key: 'left',
    node: Node { type: 'Identifier', ... name: 'a' },
    scope: Scope { ... },
    type: 'Identifier' },
  NodePath { ... key: 'right', ... type: 'Identifier' } ]
```

可以看出,referencePaths 是一个数组。在原始代码中使用的是 $a+a$,所以有两处引用,对应这里的两个成员。其中 Node 对象是引用处的 a 标识符本身。因为这是一个二项式,所以两处引用分别是 BinaryExpression 的 left 和 right,它们的父节点自然是二项式。没有兄弟节点,所以 container 是一个对象,listKey 是 undefined。

再看介绍 constantViolations。假如标识符被修改,constantViolations 中会存放所有修改该标识符节点的 Path 对象。查看 constantViolations 中的内容,原始代码中有一处修改了 a 变量,它是一个赋值表达式,left 是 a,right 是 400,如下所示:

```
constantViolations: [
  NodePath { ...
```

```
node: Node {
  type: 'AssignmentExpression', ...
  operator: '=',
  left: Node { type: 'Identifier', ... name: 'a' },
  right: Node { type: 'NumericLiteral', ... value: 400 }, ...}}
```

8.4.5 遍历作用域

scope.traverse 方法可以用来遍历作用域中的节点。可以使用 Path 对象中的 scope,也可以使用 Binding 中的 scope,推荐使用后者。来看下面这个例子:

```
traverse(ast, {
  FunctionDeclaration(path) {
    let binding = path.scope.getBinding('a');
    binding.scope.traverse(binding.scope.block, {
      AssignmentExpression(p) {
        if (p.node.left.name == 'a')
          p.node.right = t.numericLiteral(500);
      }
    });
  }
});
/*
const a = 1000;
let b = 2000;
let obj = {
  name: 'xiaojianbang',
  add: function (a) {
    a = 500;
    b = 300;
    let e = 700;
    function demo() {
      let d = 600;
    }
    demo();
    return a + a + b + 1000 + obj.name;
  }
};
obj.add(100);
*/
```

原始代码中,a=400,上述代码的作用是将它改为 a=500。假如是从 demo 这个函数入手,那么只要获取 demo 函数中的 a 的 Binding,然后遍历 binding.scope.block(也就是 a 的作用域),找到赋值表达式是 left 为 a 的,将对应的 right 改掉。

8.4.6 标识符重命名

可以使用 `scope.rename` 将标识符进行重命名,这个方法会同时修改所有引用该标识符的地方。例如将 `add` 函数中的 `b` 变量重命名为 `x`,代码如下:

```
traverse(ast, {
  FunctionExpression(path) {
    let binding = path.scope.getBinding('b');
    binding.scope.rename('b', 'x');
  }
});
/*
const a = 1000;
let x = 2000;
let obj = {
  name: 'xiaojianbang',
  add: function (a) {
    a = 400;
    x = 300;
    ...
    return a + a + x + 1000 + obj.name;
  }
};
obj.add(100);
*/
```

上述方法很方便,但是如果指定一个名字,可能会与现有标识符冲突。这时可以使用 `scope.generateUidIdentifier` 来生成一个标识符,生成的标识符不会与任何本地定义的标识符相冲突,如:

```
traverse(ast, {
  FunctionExpression(path){
    path.scope.generateUidIdentifier("uid");
    // Node { type: "Identifier", name: "_uid" }
    path.scope.generateUidIdentifier("uid");
    // Node { type: "Identifier", name: "_uid2" }
  }
});
```

使用这两种方法,就可以实现一个简单的标识符混淆方案。代码如下:

```
traverse(ast, {
  Identifier(path){
    path.scope.rename(path.node.name,
                      path.scope.generateUidIdentifier('_0x2ba6ea').name);
  }
});
```

```
/*
const _0x2ba6ea = 1000;
let _0x2ba6ea15 = 2000;
let _0x2ba6ea18 = {
  name: 'xiaojianbang',
  add: function (_0x2ba6ea14) {
    _0x2ba6ea14 = 400;
    _0x2ba6ea15 = 300;
    let _0x2ba6ea9 = 700;
    function _0x2ba6ea12() {
      let _0x2ba6ea11 = 600;
    }
    _0x2ba6ea12();
    return _0x2ba6ea14 + _0x2ba6ea14 + _0x2ba6ea15 + 1000 + _0x2ba6ea18.name;
  }
};
_0x2ba6ea18.add(100);
//在控制台执行以上代码,最后输出'2100xiaojianbang'
*/
```

实际上标识符混淆方案还可以做得更复杂。例如,上述代码中,如果再多定义一些函数,会发现各函数之间的局部变量名是不重复的。假如把各个函数之间的局部变量定义成重复的,甚至还可以让函数中的局部变量跟当前函数中没有引用到的全局变量重名,原始代码中的全局变量 `a` 与 `add` 中的 `a` 参数。更复杂的标识符混淆方案将在第 9 章详细介绍。

8.4.7 scope 的其他方法

1. scope.hasBinding('a')

该方法查询是否有标识符 `a` 的绑定,返回 `true` 或 `false`。可以用 `scope.getBinding('a')` 代替,`scope.getBinding('a')` 返回 `undefined`,等同于 `scope.hasBinding('a')` 返回 `false`。

2. scope.hasOwnBinding('a')

该方法查询当前节点中是否有自己的绑定,返回 `true` 或 `false`。例如,对于 `demo` 函数, `OwnBinding` 只有一个 `d`。函数名 `demo` 虽然也是标识符,但不属于 `demo` 函数的 `OwnBinding` 范畴,是属于它的父级作用域的,如:

```
traverse(ast, {
  FunctionDeclaration(path){
    console.log( path.scope.parent.hasOwnBinding('demo') );
  }
});
// true
```

同样可以使用 `scope.getOwnBinding('a')` 代替它, `scope.getOwnBinding('a')` 返回 `undefined`, 等同于 `scope.hasOwnBinding('a')` 返回 `false`。原始代码中 `scope.hasOwnBinding('a')` 也是通过 `scope.getOwnBinding('a')` 来实现的。

3. `scope.getAllBindings`

该方法获取当前节点的所有绑定, 会返回一个对象。该对象以标识符名为属性名, 对应的 `Binding` 为属性值, 代码如下:

```
traverse(ast, {
  FunctionDeclaration(path){
    console.log( path.scope.getAllBindings() );
  }
});
/*
[Object: null prototype]{
  d: Binding {...},
  a: Binding {...},
  demo: Binding {...},
  e: Binding {...},
  b: Binding {...},
  obj: Binding {...}
}
*/
```

4. `scope.hasReference('a')`

该方法查询当前节点中是否有 `a` 标识符的引用, 返回 `true` 或 `false`。

5. `scope.getBindingIdentifier('a')`

该方法获取当前节点中绑定的 `a` 标识符, 返回 `Identifier` 的 `Node` 对象。同样, 这个方法也有 `Own` 版本, 为 `scope.getOwnBindingIdentifier('a')`。

8.5 小结

本章详细介绍了 AST 的基本结构, 了解 AST 结构是学习后续知识的基础。接着讲述了 `traverse` 组件与 `visitor` 的使用, 这在 Babel 自动化处理 JS 代码过程中必不可少。然后阐明了 `types` 组件的用法, 如果要生成新的节点, 这个组件最合适。如果要修改节点或者将生成的新节点加入到已有的节点中, 那么 `Path` 对象相关的方法必不可少。`Path` 对象是本章的重点。另外, 标识符都具有自己的作用域, 不同作用域中可能会有相同名字的标识符, 因此在处理代码的过程中, 要格外小心这种情况, 选择在作用域内遍历节点更合理, Babel 中的 `scope` 是处理这种情况的“利器”。

Babel 中的 API 还有很多没有介绍, 剩余的需要自己去学习。一定得多看、多练和多思考, 才能更快地掌握这些 API 的使用。

8.6 习题

1. 将以下代码放到 AST Expoer 网站中解析,并分析解析以后的结构。

```
function test(a){  
    return a + 1000;  
}
```

2. 使用 types 组件生成第 1 题中的代码。
3. 以第 1 题的代码为源代码,使用 Babel 的 API 将其改为如下形式。

```
function test(a){  
    return function(){  
        return a + 1000;  
    }();  
}
```

4. 以第 3 题的代码为源代码,使用 Babel 的 API 将其改为如下形式。

```
function test(a, b){  
    b = 2000;  
    return function(){  
        return a + b, a + 1000;  
    }();  
}
```

5. 以第 4 题的代码为源代码,使用 Babel 的 API 将其中的变量 a 改为 x。
6. 编写一个 visitor,判断标识符是否为当前参数。